

# Programação em R

Copyright: Carlos Cinelli

Julho, 2016

# Introdução e Motivação

## O que é o R?

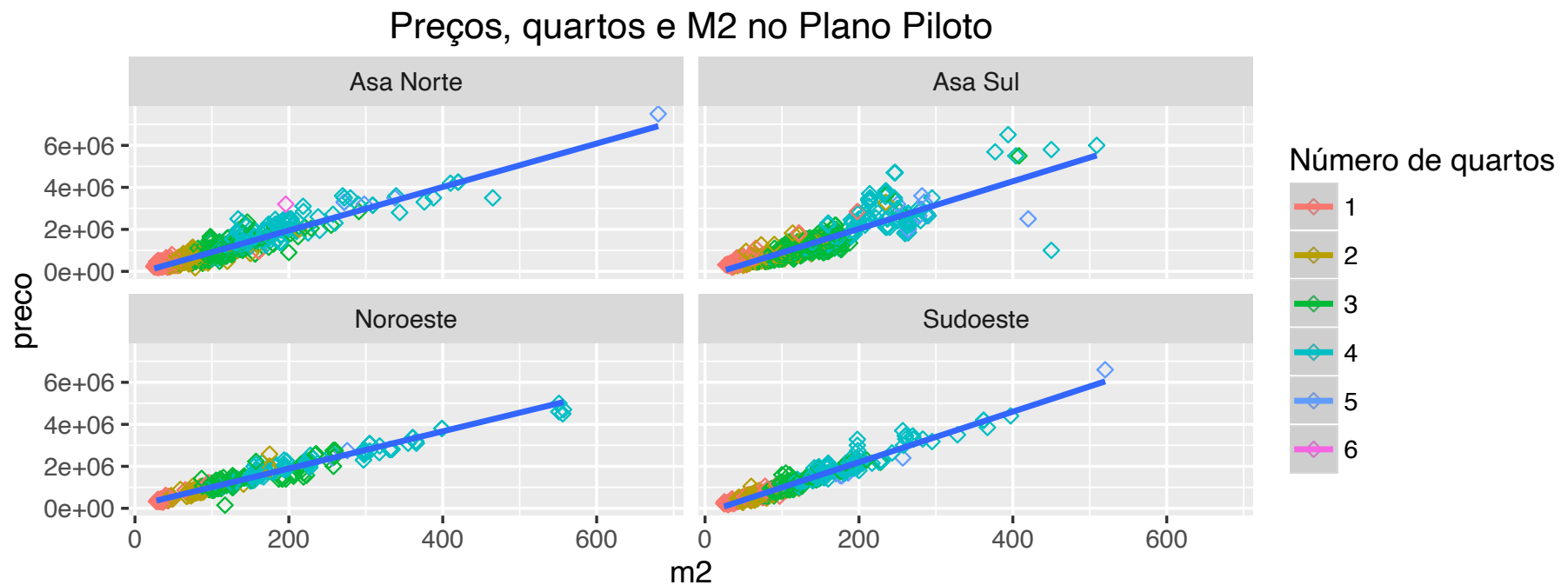
- O R é uma linguagem de programação com foco em análise de dados;
- Criado na Nova Zelândia por dois estatísticos: Ross Ihaka e Robert Gentleman;
  - Baseado na linguagem S, desenvolvida por John Chambers (e colegas) na Bell Laboratories;
- É uma linguagem de programação interpretada, voltada à interação dinâmica com os dados e modelos.

## Por que o R?

- O R é gratuito e de código aberto;
- Compatível com todas as plataformas (Windows, Mac, Linux);
- É mais do que um software estatístico:
  - Ambiente que permite explorar dados interativamente; mas, à medida que a análise evolui, é uma linguagem de programação completa para desenvolver e automatizar soluções, desenvolver software (pacotes);
  - Ferramenta poderosa para manipulação, processamento, visualização e análise de dados, bem como simulações e modelagem estatísticas.
- Comunidade grande que contribui ativamente, com pessoas tanto do mercado (Google, AT&T, empresas de investimento e finanças) quanto da academia (professores de diversas universidades):
  - Mais de 8000 pacotes gratuitos e abertos para download.

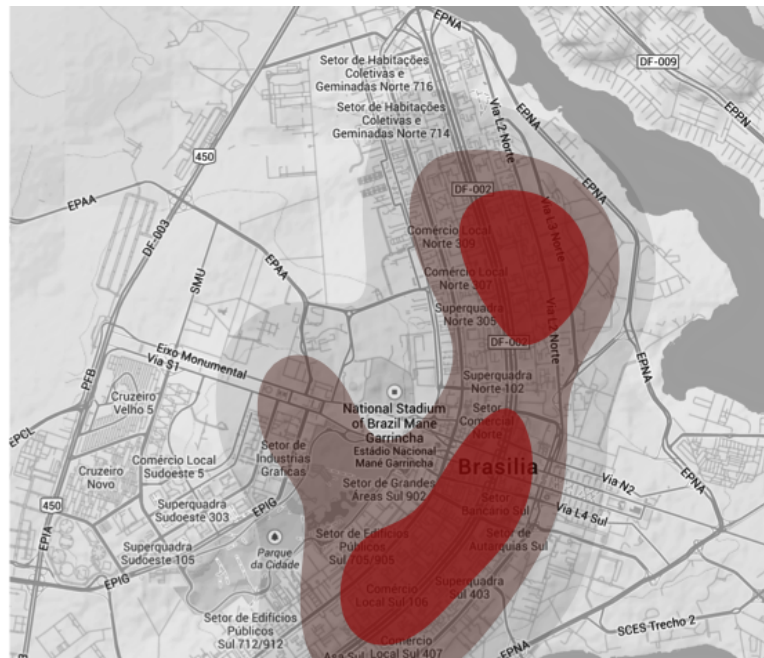
## Alguns exemplos

**Gráficos e estatística:** gráfico de dispersão com preço contra metro quadrado por bairro, cor dos pontos de acordo com número de quartos e linha de regressão.



## Alguns exemplos

**Mapas e estatística:** Webscraping de dados de roubos e download do mapa de Brasília no Google Maps; manipulação dos dados e construção de gráficos de calor geolocalizados.



## Alguns exemplos

**Apresentações e documentos de suas análises:** Integração com *markdown*,  $\text{\LaTeX}$ , HTML + JavaScript e Word, o que permite a elaboração de *papers*, relatórios, apresentações de maneira rápida e conjugada à análise. Esta e as demais apresentações deste curso, bem como as listas de exercícios, foram todas feitas no R.

## Sobre os slides

Os slides deste curso apresentam códigos em R com o resultado esperado. Você deve **acompanhar ativamente**, digitando (ou, em último caso, copiando e colando) os códigos. Programação não se aprende somente olhando para a apresentação! Também serão realizados exercícios para consolidação do conteúdo.

```
x <- 10; y <- 20;  
x == y
```

CÓDIGO EM R.

```
# [1] FALSE
```

RESULTADO ESPERADO DO CÓDIGO ACIMA

```
x <- c(10, 20, 30); y <- c(10, 10, 30)  
x == y
```

```
# [1] TRUE FALSE TRUE
```

## Sobre os exercícios

- Durante a apresentação de cada tópico faremos exercícios entre os slides. Os exercícios serão feitos com tempo controlado e todas soluções serão apresentadas. Não se preocupe se não conseguir fazer todos durante o tempo controlado.
- Também teremos listas de exercícios. Como a turma é heterogênea, os exercícios da lista variam em nível de dificuldade e você não precisa conseguir fazer todos - a idéia é justamente ter exercícios tanto para consolidar conhecimentos quanto para desafiar um pouco quem for pegando o conteúdo mais rapidamente.
- Vocês receberão a resolução de todos os exercícios da lista também, entretanto, para resolução “ao vivo” em sala de aula, priorizaremos aqueles mais importantes, a depender do andamento das aulas.

# R e Rstudio

## Instalação

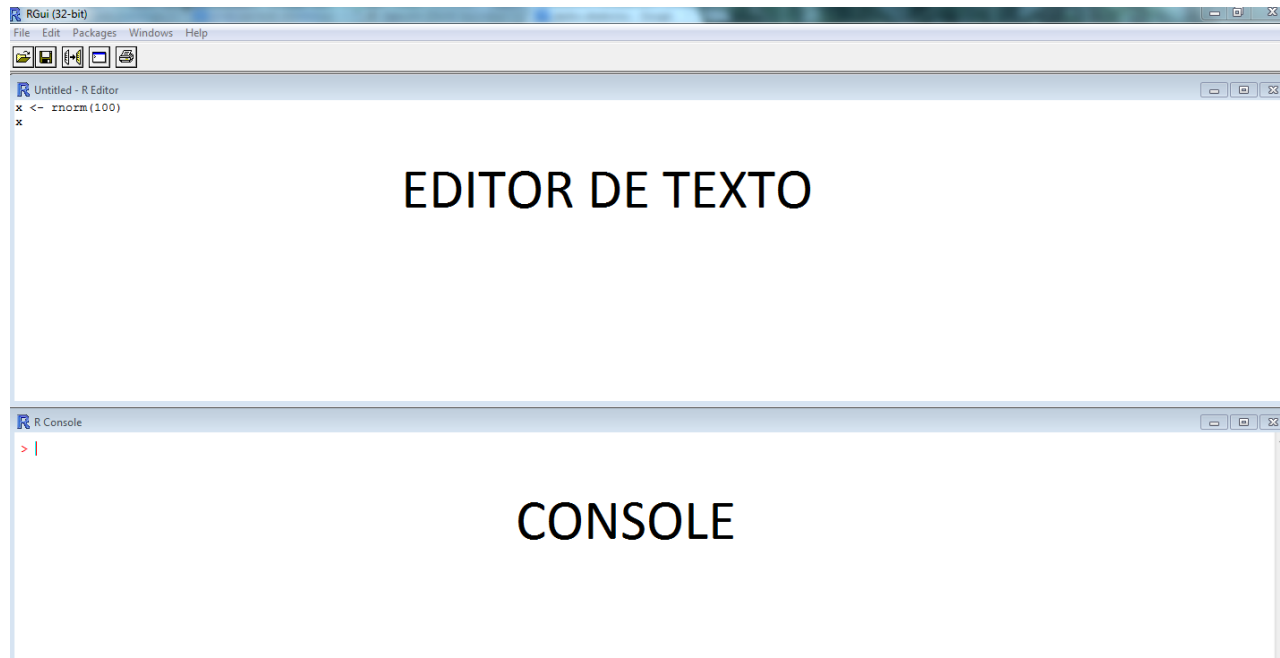
O R e RStudio já vêm com distribuições compiladas para Windows, Mac e Linux. A instalação é bastante fácil e em geral você apenas tem que seguir as instruções da tela. Para ter um ambiente completo de desenvolvimento no R, certifique-se de ter em sua máquina:

- a versão mais recente do R;
- a versão mais recente do RStudio (IDE que vamos usar);
- MikTeX (Win) ou MacTeX (Mac) para relatórios em  $\text{\LaTeX}$ ;
- RTools (Win) ou Xcode com command line tools (Mac), para criar pacotes, usar C++ etc.

Os slides a seguir tomarão como base, em geral, as teclas de atalho do Windows.

## R Gui

Ao instalar o R, automaticamente também é instalada uma interface gráfica chamada R Gui. Abra o R no seu computador. A primeira tela que você verá é a do console. Abra também uma tela de editor de texto em “file” -> “new script”.



## R Gui

Escreva os seguintes comandos no editor de texto e aperte “ctrl+R”.

```
1+1
```

O comando será enviado para o console. Agora faça o seguinte gráfico:

```
plot(1:10)
```

Uma nova tela será aberta com o gráfico. Saia do R Gui escrevendo q() no console ou apertando “alt+f4”. Ele irá perguntar se você deseja salvar o trabalho e o script. Clique em “não” para os dois casos. Estamos saindo do R Gui pois iremos trabalhar no RStudio.

## RStudio

Apesar de o R vir com uma interface gráfica bem interessante, existe um IDE (Integrated Development Environment - Ambiente de Desenvolvimento Integrado) chamado RStudio, com várias funcionalidades e gratuito. No decorrer de nossas aulas iremos utilizar somente o RStudio.

O RStudio tem algumas vantagens em relação ao R Gui:

- Highlight do código;
- Autocomplete;
- Match automático de parenteses e chaves;
- Interface intuitiva para objetos, gráficos, script;
- Projetos (com controle de versão);
- Interação com HTML, entre outras.

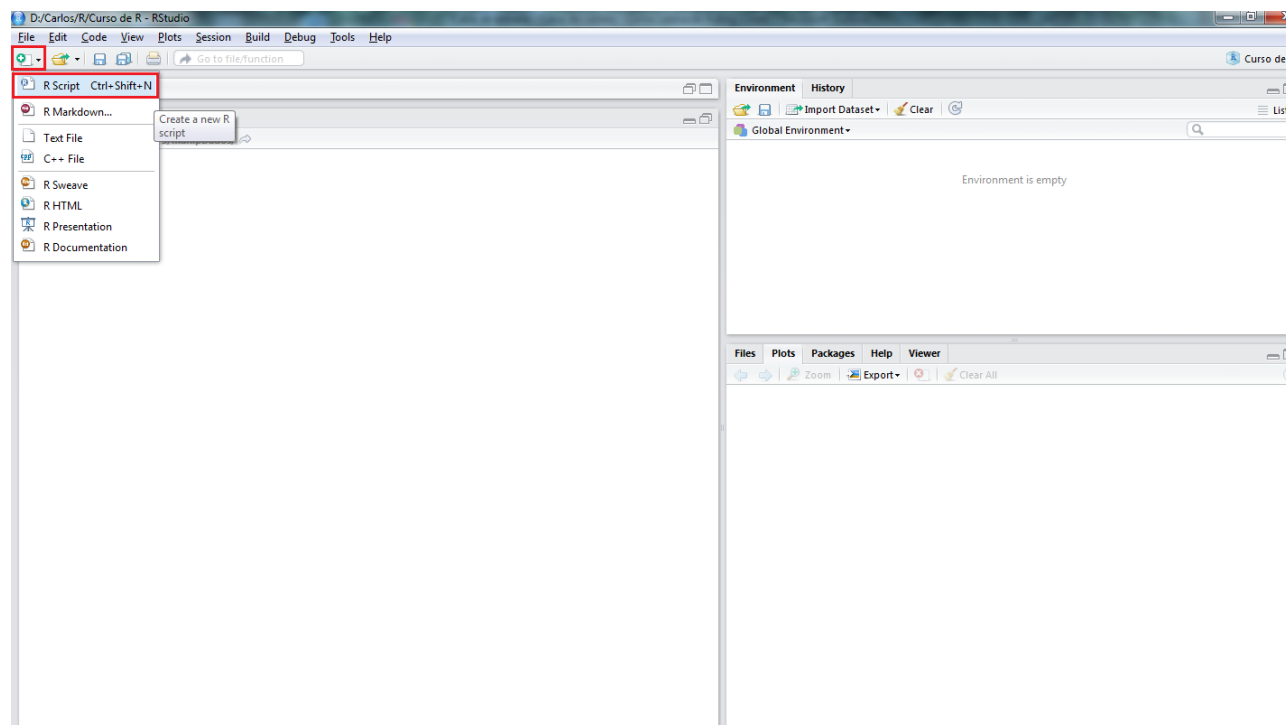
## Projetos no RStudio

Vamos criar um projeto do RStudio para o nosso curso de R. Esta é uma maneira simples e fácil de gerenciar os scripts, dados e demais documentos relacionados a um projeto de R em que você esteja trabalhando. Paremos agora para:

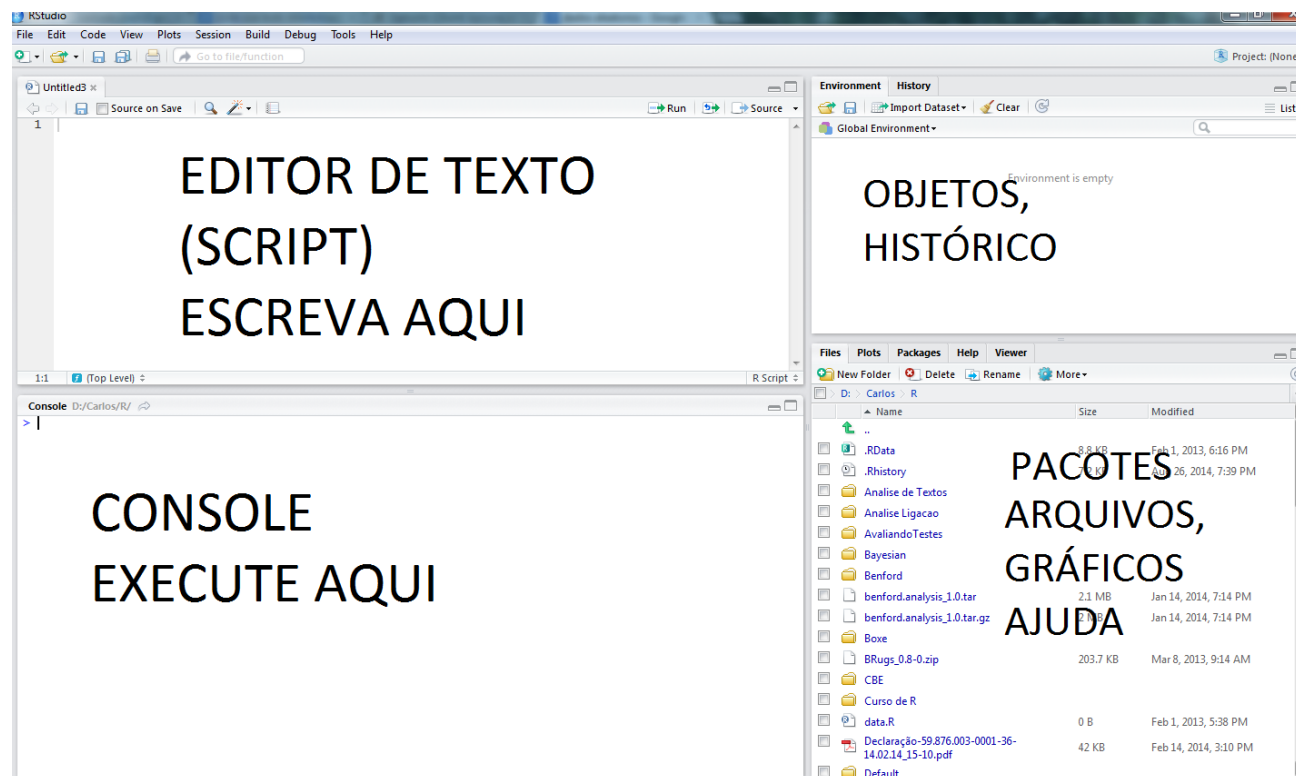
- Criar projeto de RStudio em uma pasta nova;
- Criar uma pasta de “Dados” para guardar algumas bases de dados utilizados em aula;
- Criar uma pasta de slides para guardar os slides (inclusive este) das aulas.

# RStudio

Inicie um novo Script em “File” -> “New File” -> “New RScript”. Você também pode fazer isso com **“CTRL + SHIFT + N”** ou acessando o botão abaixo.



# RStudio



## RStudio

- **Script:** A tela superior esquerda do RStudio é o editor de texto onde você vai escrever seus Scripts. Ele possui code highlighting entre outras funcionalidades.
- **Console:** No canto inferior esquerdo fica o console. O console nada mais é do que uma seção aberta de R, em que os comando são executados.
- **Área de trabalho e histórico:** Ficam no canto superior direito. Os objetos criados na seção e o histórico dos comandos podem ser acessados ali.
- **Arquivos, Gráficos, Pacotes, Ajuda:** Ficam no canto inferior direito. Você pode explorar pastas e arquivos diretamente do RStudio na aba “Files”; os gráficos que forem feitos apareceram na aba “Plots”. Os pacotes instalados em sua máquina estão listados em “Packages”. As ajudas das funções aparecem em “Help”. E o “Viewer” serve para visualização de imagens/páginas em HTML e JavaScript.

## Comandos pelo Script

Acostume-se a escrever o código no Script ao invés de ficar escrevendo diretamente no console.

Escreva o código abaixo no Script:

```
1+1
```

E aperte “ctrl” + “enter”. Isso envia o comando para o console e o resultado é exibido logo abaixo.

```
1+1  
## [1] 2
```

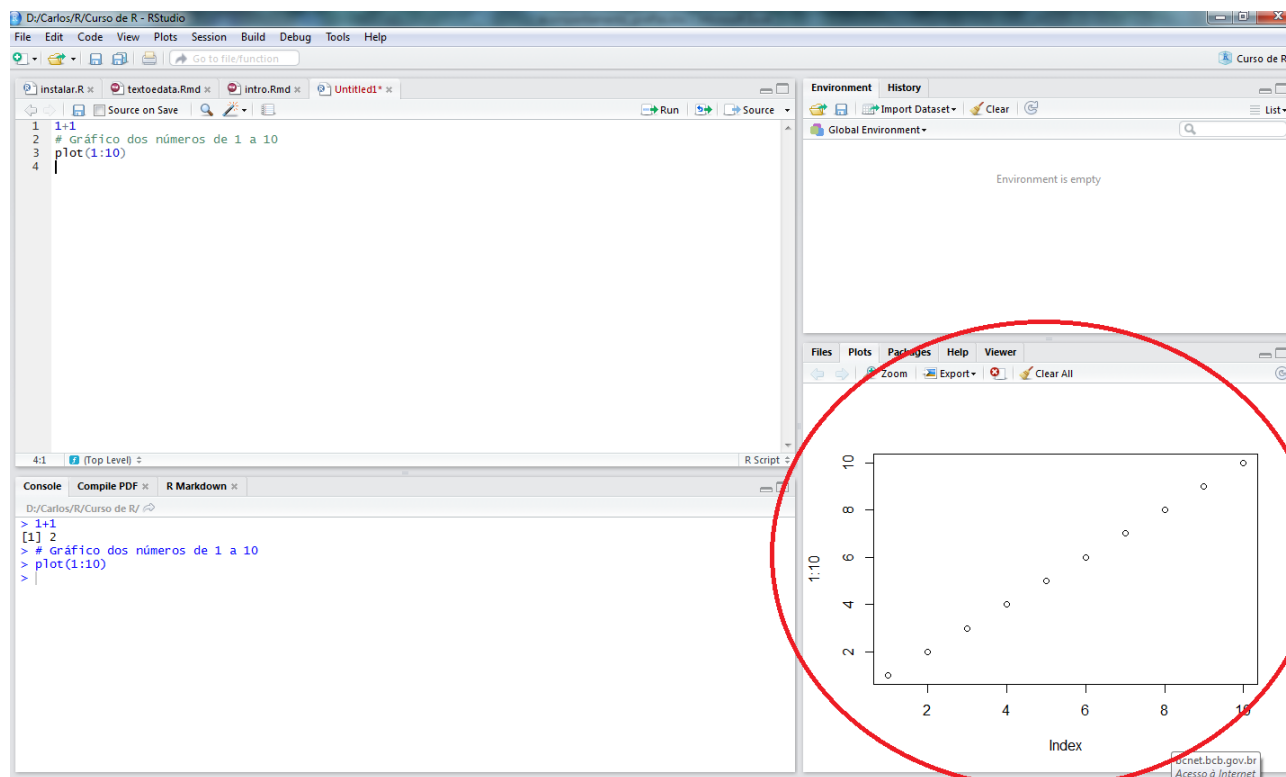
## Comandos pelo Script

Agora escreva o seguinte código no Script.

```
# Gráfico dos números de 1 a 10  
plot(1:10)
```

O primeiro comando “*# Gráfico dos números de 1 a 10*” é, na verdade, um comentário. Comentários em Scripts do R são seguidos do símbolo #, e tudo que estiver após # não será executado. É uma boa prática comentar seu código, para que ele seja de fácil manutenção, tanto para você mesmo quanto para outros colegas. O segundo comando é de um gráfico. Aperte “ctrl” + “enter” nas duas linhas. O gráfico aparecerá no canto inferior direito do RStudio.

## Comandos pelo Script

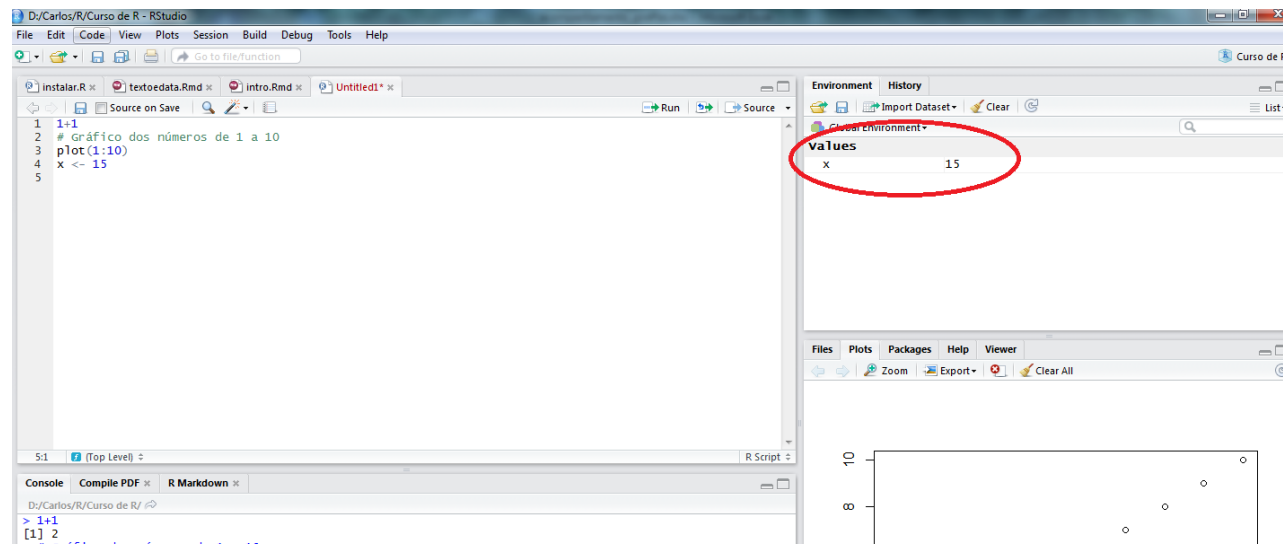


## Comandos pelo Script

Agora digite o seguinte comando no editor e aperte “ctrl” + “enter”:

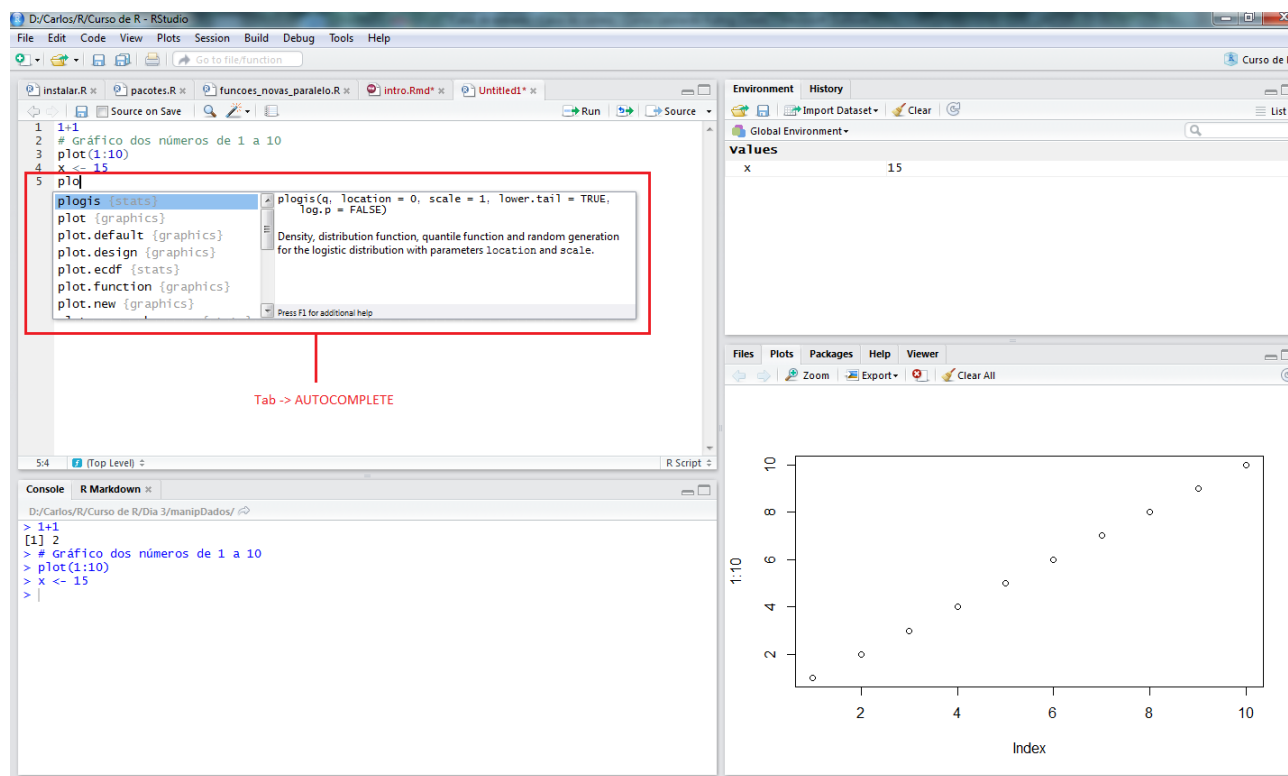
```
x <- 15
```

Atribuímos o valor 15 à variável x. Note que isso aparece no canto superior direito do RStudio.



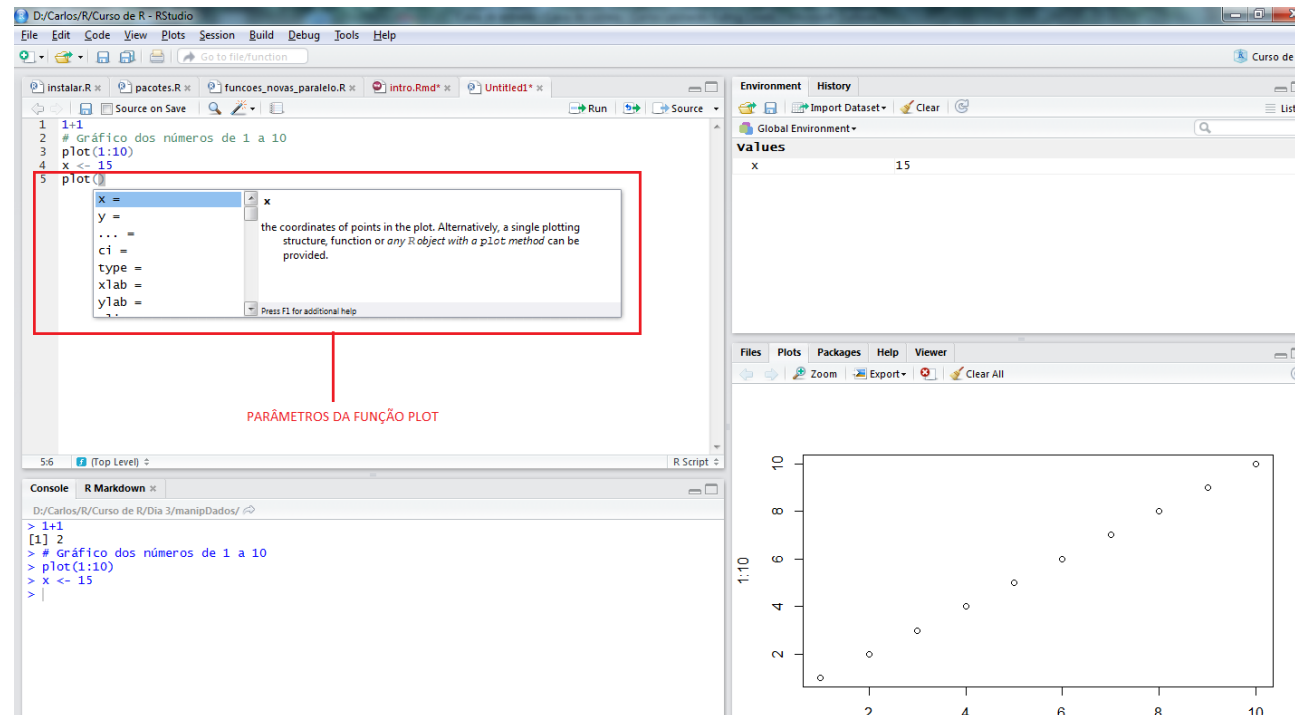
## Comandos pelo Script

O RStudio tem autocomplete. Escreve apenas plo e aperte **Tab**.



## Comandos pelo Script

Isso também funciona dentro da função, para vermos os parâmetros disponíveis. Escreva `plot()` e aperte **Tab**.



## Comandos pelo Script

Outras teclas de atalho:

- **CTRL + 1**: Passa o cursor para o Script;
- **CTRL + 2**: Passa o cursor para o console;
- **SETA PARA CIMA (no console)**: acessa o histórico de comandos anteriores.

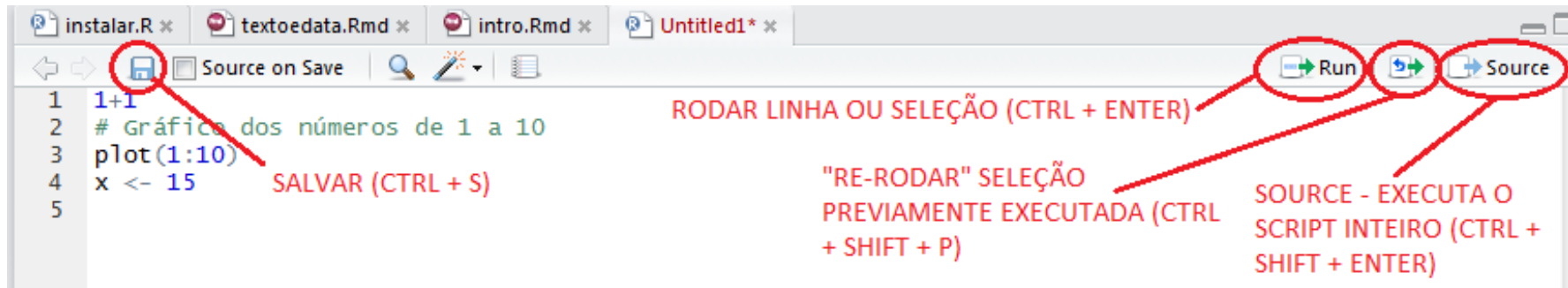
Abra um novo Script:

- **CTRL + ALT + SETA PARA ESQUERDA OU DIREITA**: Navega entre as abas de script abertas.

## Comandos pelo Script

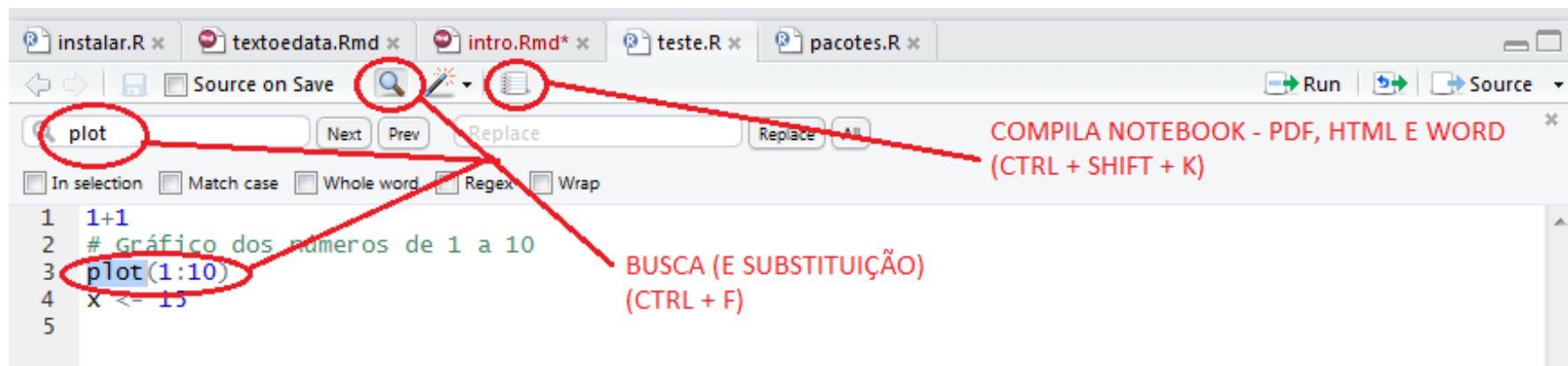
Outras teclas de atalho:

- **CTRL + SHIFT + P**: “Previous command”, roda o último comando executado;
- **CTRL + SHIFT + ENTER**: “Source”. Executa o Script inteiro;
- **CTRL + S**: Salva o Script;



## Comandos pelo Script

- **CTRL + L**: Limpa o console;
- **CTRL + F**: Busca (e substituição). Aceita REGEX;
- **CTRL + SHIFT + K**: Compila “Notebook” em PDF, HTML ou Word;
- **ALT + SHIFT + K**: Veja os outros atalhos.
- Veja mais dicas no Cheat Sheet do RStudio (<http://www.ibpad.com.br/wp-content/uploads/2016/04/rstudio-IDE-cheatsheet.pdf>).



## Ajuda - Função Específica

?mean; help(mean)

The screenshot displays the RStudio interface. The script editor on the left contains the following code:

```
1 1+1
2 # Gráfico dos números de 1 a 10
3 plot(1:10)
4 x = 15
5 ?mean
6 help(mean)
```

A red circle highlights the `?mean` and `help(mean)` lines. A red arrow points from this circle to the console output below, which shows the execution of these commands:

```
> 1+1
[1] 2
> # Gráfico dos números de 1 a 10
> plot(1:10)
> x = 15
> ?mean
> help(mean)
```

Another red circle highlights the `?mean` and `help(mean)` lines in the console output. A red arrow points from this circle to the right-hand pane, which displays the help documentation for the `mean` function. The documentation includes the title "Arithmetic Mean", a description, usage instructions, and arguments.

**Arithmetic Mean**

**Description**

Generic function for the (trimmed) arithmetic mean.

**Usage**

```
mean(x, ...)
```

**Arguments**

- `x` An R object. Currently there are methods for numeric/logical vectors and [date](#), [date-time](#) and [time interval](#) objects. Complex vectors are allowed for `trim = 0`, only.
- `trim` the fraction (0 to 0.5) of observations to be trimmed from each end of `x` before

## Ajuda - Função Específica

mean (base) R Documentation

Arithmetic Mean

Description

Generic function for the (trimmed) arithmetic mean.

Usage

```
mean(x, ...)
```

```
## Default S3 method:  
mean(x, trim = 0, na.rm = FALSE, ...)
```

Arguments

**x** An R object. Currently there are methods for numeric/logical vectors and [date](#), [date-time](#) and [time interval](#) objects. Complex vectors are allowed for `trim = 0`, only.

**trim** the fraction (0 to 0.5) of observations to be trimmed from each end of `x` before the mean is computed. Values of `trim` outside that range are taken as the nearest endpoint.

**na.rm** a logical value indicating whether NA values should be stripped before the computation proceeds.

**...** further arguments passed to or from other methods.

Value

If `trim` is zero (the default), the arithmetic mean of the values in `x` is computed, as a numeric or complex vector of length one. If `x` is not logical (coerced to numeric), numeric (including integer) or complex, `NA_real_` is returned, with a warning.

If `trim` is non-zero, a symmetrically trimmed mean is computed with a fraction of `trim` observations deleted from each end before the mean is computed.

References

Becker, R. A., Chambers, J. M. and Wilks, A. R. (1988) *The New S Language*. Wadsworth & Brooks/Cole.

See Also

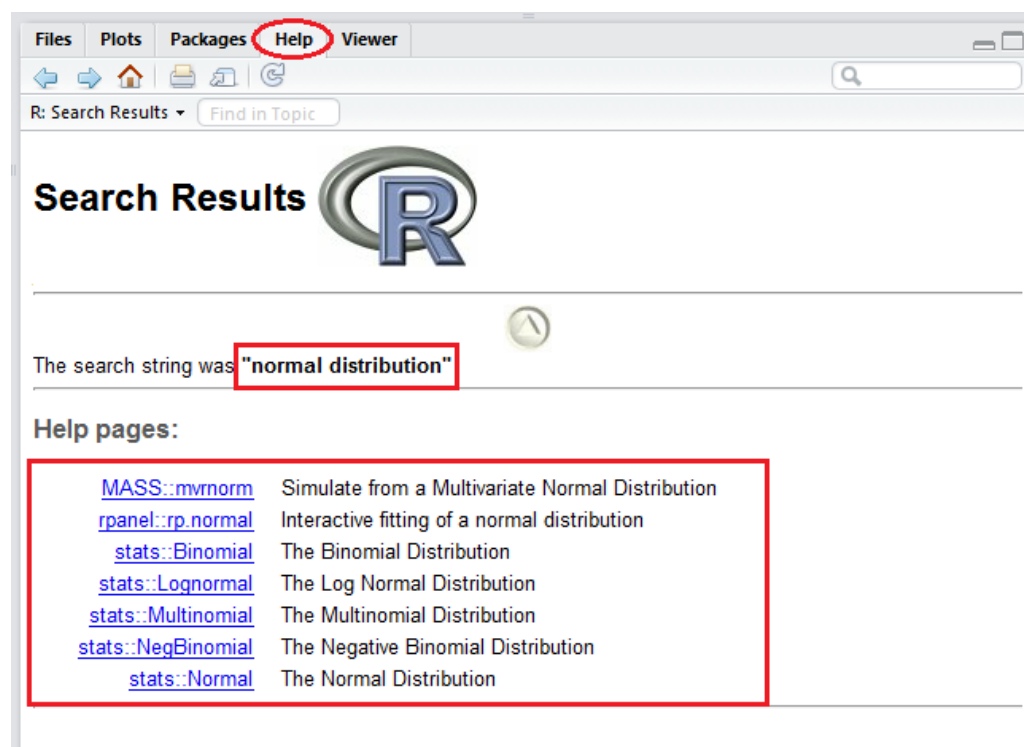
[weighted.mean](#), [mean.POSIXct](#), [colMeans](#) for row and column means.

Examples

```
x <- c(0:10, 50)  
xm <- mean(x)  
c(xm, mean(x, trim = 0.10))
```

## Ajuda - Busca por String

```
??"normal distribution"; help.search("normal distribution")
```



## Ajuda - Internet

- StackOverflow (em Inglês e Português)

The screenshot shows the StackOverflow website interface. At the top, there's a navigation bar with links for Questions, Tags, Users, Badges, and Unanswered. A prominent red box highlights the '66,638 questions tagged' for the 'r' tag. Below this, the 'Tagged Questions' section lists several questions with their respective votes, answers, and views. The questions are:

- How to plot the output of a multivariate regression using GGLOT**: I have a regression with fixed effects/other covariates and I want to plot the outcome and the predictor variable of interest after controlling for the fixed effects. So, I want to plot a curve that ...
- Where are the values of this variable coming from? Doesn't seemed to be defined anywhere**: I am new to Rstudio and I'm trying to learn how to use R Markdown. When I create a new R Markdown file, I am greeted with the following default document. http://i.stack.imgur.com/HH0MY.png Note how ...
- Want individual components as subsets in a stacked bar chart**: I am trying to create a stacked bar plot which will show the revenue of the company and various components of its cost of sales (operating expenses, other fixed costs etc.). Now I want the individual ...
- Factor NA not plotting**: I am working through Ch 6 in Medley's book (Elementary Graphics for Data Analysis). Specifically, I am ...

On the right side, there's a 'Featured on Meta' section with links to 'An experiment: Stack Overflow TV' and 'Hot Meta Posts'. Below that, a 'Related Tags' section lists various tags like ggplot2, plot, data.frame, matrix, data.table, statistics, list, loops, function, and time series.

## Ajuda - Internet

- **Grupos de e-mail:** [www.r-project.org/mail.html](http://www.r-project.org/mail.html)
  - **R Help:** lista de e-mails principal, para dúvidas gerais.
  - **R Announce:** Lista de anúncios sobre desenvolvimento do R.
  - **R Packages:** Lista de anúncios sobre pacotes do R.
  - **R Devel:** Lista de discussão de desenvolvedores no R.
- **Blogs**
  - **R Bloggers:** consolidador de blogs sobre R (em inglês).
  - **Em Português:**
    - Análise Real (<http://analisereal.com>) com livro em elaboração (<http://analisereal.com/introducao-a-analise-de-dados-com-r/>) ;
    - Sociais e Métodos (<https://sociaisemetodos.wordpress.com>);
    - Dados Aleatórios (<http://www.dadosaleatorios.com.br/>) entre outros.

## Pacotes

A principal forma de distribuição de códigos no R é por meio de pacotes. Um pacote pode ser entendido como um conjunto de códigos auto-contido que adiciona funcionalidades ao R.

Para carregar um pacote, use a função `library()`.

Ao carregar um pacote, você está adicionando suas funções ao `search` da seção, permitindo que você chame estas funções diretamente. Por exemplo, a função `mvrnorm`, que gera números aleatórios de uma normal multivariada, está no pacote MASS.

```
Sigma <- matrix(c(10,3,3,2), nrow = 2, ncol = 2) # Matriz de Var-Covar
mu <- c(1, 10) # Médias
x <- mvrnorm(n = 100, mu, Sigma) # Tenta gerar 100 obs, mas dá erro
## Error in eval(expr, envir, enclos): não foi possível encontrar a função "mvrnorm"

library(MASS) # Carrega pacote
x <- mvrnorm(n = 100, mu, Sigma) # Agora funciona
```

## Pacotes

Para ver o que está no search do R, utilize a função `search()`. Note que o pacote MASS agora esta lá.

```
search()  
## [1] ".GlobalEnv"      "package:MASS"      "package:ggplot2"  
## [4] "package:stats"    "package:graphics"  "package:grDevices"  
## [7] "package:utils"    "package:datasets"  "package:methods"  
## [10] "Autoloads"        "package:base"
```

Para descarregar um pacote, utilize a função `detach()`.

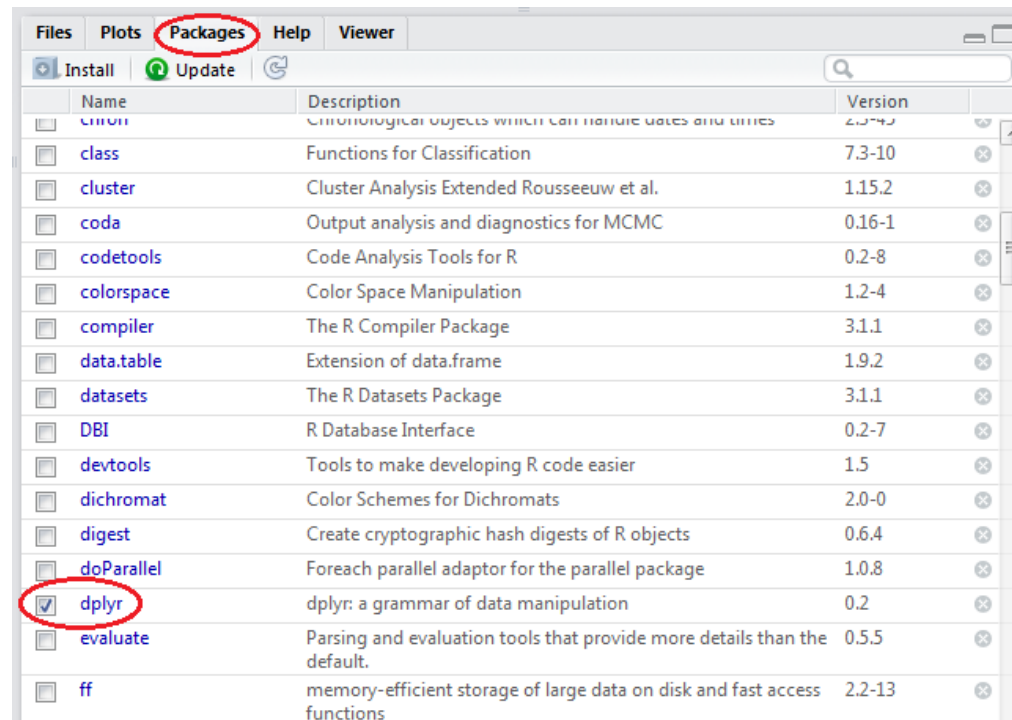
```
detach(package:MASS)
```

Às vezes pacotes tem o mesmo nome de funções. Neste caso, se ambos forem carregados, a função que prevalece é a do pacote que foi carregado por último. Uma outra forma de resolver isso é usar o nome do pacote e o operador `::` antes de chamar a função. Neste caso não há ambiguidade.

```
x <- MASS::mvrnorm(n = 100, mu, Sigma)
```

## Pacotes

Você também pode carregar ou descarregar pacotes pelo menu do canto inferior direito do RStudio, clicando na caixa ao lado do nome do pacote.



## Pacotes

Para instalar um pacote, use a função `install.packages()`. Por exemplo, o pacote `dplyr` é muito utilizado para manipulação de dados. Para instalá-lo e desinstalá-lo, temos os seguintes comandos.

```
install.packages("dplyr")  
  
##### Não rode #####  
remove.packages(dplyr)
```

## Pacotes, CRAN e Sites

Grande parte dos pacotes do R estão centralizados em um repositório chamado CRAN (The Comprehensive R Archive Network), com diversos espelhos ao redor do mundo. Agora, vamos explorar um pouco o site oficial do R, **[www.r-project.org](http://www.r-project.org)**, e do CRAN, **[cran.r-project.org](http://cran.r-project.org)**.

- Manuais, livros, documentação;
- Listas de e-mails;
- The R Journal;
- Lista de pacotes;
- Task Views.

Introdução à área de Trabalho

O objeto básico do R: vetores

Vetorização, reciclagem, operadores matemáticos, relacionais e lógicos básicos

# Introdução à área de Trabalho

## Assignment Operator

No R, praticamente tudo é um objeto. Estes objetos podem ser variáveis, vetores, matrizes, arrays, números, caracteres e, inclusive, funções. Durante o uso do R, você irá criar e guardar estes objetos com um nome, por exemplo:

```
x1 <- 10 # atribui o valor 10 à variável x1
x1
## [1] 10
```

O operador <- é conhecido como “assignment operator” e, no caso acima, atribuiu o valor 10 a um objeto de nome x1. Na maior parte das vezes, o operador = é equivalente a <-. Vejamos:

```
x2 = 20 # atribui o valor 20 à variável x2
x2
## [1] 20
```

## Assignment Operator

Entretanto, recomenda-se o uso do `<-` pois ele funciona sempre como atribuição, enquanto que o operador `=` é usado em parâmetros dentro de uma função e o comportamento é diferente (veremos isso mais a frente).

O assignment operator `<-` nada mais é do que uma função. Uma função equivalente ao `<-`, que tem sintaxe mais usual, é a função `assign("nomeObjeto", valorObjeto)`.

```
assign("x3", 4) # cria a variável x3 com o valor 4
```

```
x3
```

```
## [1] 4
```

```
x3 <- 4 # note que é equivalente!
```

```
x3
```

```
## [1] 4
```

## Assignment Operator

Uma vez que você atribuiu uma variável a um nome, você pode utilizá-la em operações e funções utilizando seu nome.

```
x1 + x2 + x3 # soma x1,x2,x3  
## [1] 34
```

```
x1/x2 # divide x1 por x2  
## [1] 0.5
```

```
x1 * x2 # multiplica x1 por x2  
## [1] 200
```

```
y <- x1*x2 + x3 #cria y com o resultado  
y  
## [1] 204
```

## Nomes de variáveis

O R é *case sensitive* (diferentemente do SQL, por exemplo), portanto x1 e X1 são objetos diferentes.

```
X1 # Não vai encontrar
```

```
## Error in eval(expr, envir, enclos): objeto 'X1' não encontrado
```

```
x1 # Encontra
```

```
## [1] 10
```

## Nomes de variáveis

Nomes de variáveis no R podem conter combinações arbitrárias de números, textos, bem como ponto (.) e *underscore* (\_). Entretanto, os nomes não podem **começar** com números ou *underscore*.

```
a1_B2.c15 <- "variável com nome estranho"
a1_B2.c15
## [1] "variável com nome estranho"
```

```
_vai_dar_erro <- 2000
## Error: <text>:1:1: unexpected input
## 1: _
##    ^
```

## Encontrando e removendo objetos: `ls()`

Para listar todos os objetos que estão na sua área de trabalho, você pode usar a função `ls()`.

```
ls()  
## [1] "a1_B2.c15" "x1"          "x2"          "x3"          "y"
```

Note que apareceram todos os objetos que criamos.

## Encontrando e removendo objetos: `rm()`

A função `rm(objeto)` remove um objeto da área de trabalho.

```
rm(x3) # remove o objeto x3 da área de trabalho
```

```
x3 # não encontrará o objeto
```

```
## Error in eval(expr, envir, enclos): objeto 'x3' não encontrado
```

```
ls() # note que x3 não aparecerá na lista de objetos
```

```
## [1] "a1_B2.c15" "x1" "x2" "y"
```

## Salvando e carregando objetos na área de trabalho

Para salvar uma cópia de todos os objetos criados você pode utilizar a função `save.image()`:

```
# salva a área de trabalho no arquivo "aula_1.RData"  
save.image(file = "aula_1.RData")
```

Introdução à área de Trabalho

**O objeto básico do R: vetores**

Vetorização, reciclagem, operadores matemáticos, relacionais e lógicos básicos

## O objeto básico do R: vetores

## Vetores: concatenação

No R, a estrutura mais simples de dados é um vetor. Os objetos `x1`, `x2` e `x3` são, na verdade, vetores de tamanho 1. Para construir um vetor você pode utilizar a função `c()`, que concatena uma quantidade arbitrária de objetos.

```
# concatena x1, 2, 3 e x2.
```

```
x3 <- c(x1, 2, 3, x2)
```

```
x3
```

```
## [1] 10  2  3 20
```

```
# concatena "a", "b" e "c"
```

```
x4 <- c("a", "b", "c")
```

```
x4
```

```
## [1] "a" "b" "c"
```

## Classes de vetores: numeric, integer, complex, character

Os vetores no R podem ser, entre outros, de tipos: *numeric* (número comum), *integer* (inteiro), *complex* (número complexo), *character* (texto), *logical* (lógicos, booleanos). Também há datas e fatores, que discutiremos mais a frente.

```
numero <- c(546.90, 10, 789)

# notem o L
inteiro <- c(1L, 98L)

# notem o i
complexo <- c(20i, 2 + 9i)

# notem as aspas
texto <- c("aspas duplas", 'aspas simples', "aspas 'dentro' do texto")

# sempre maiúsculo
logico <- c(TRUE, FALSE, TRUE)
```

## Classes de vetores: `class()`

A função `class()` é útil para identificar a classe de um objeto.

```
class(numero)
## [1] "numeric"

class(inteiro)
## [1] "integer"

class(complexo)
## [1] "complex"

class(texto)
## [1] "character"

class(logico)
## [1] "logical"
```

## Classes de vetores: `is.xxx`

Você pode testar se um vetor é de determinada classe com as funções `is.xxx` (sendo “xxx” a classe). Por exemplo:

```
is.numeric(numero)
## [1] TRUE

is.character(numero)
## [1] FALSE

is.character(texto)
## [1] TRUE

is.logical(texto)
## [1] FALSE
```

## Coerção: vetores tem somente uma única classe!

Um vetor somente pode ter elementos de uma única classe. Não é possível, por exemplo, misturar textos com números.

### Coerção

Quando você mistura elementos de classes diferentes em um vetor, o R faz a coerção do objeto para uma das classes, obedecendo a seguinte ordem de prioridade:

- lógico < inteiro < numérico < complexo < texto

## Coerção: vetores tem somente uma única classe!

Vejamos alguns exemplos:

```
x <- c(1, 2, 3L)
class(x)
## [1] "numeric"
```

```
x <- c(1, 2, 3L, 4i)
class(x)
## [1] "complex"
```

```
x <- c(1, 2, 3L, 4i, "5")
class(x)
## [1] "character"
```

## Coerção: `as.xxx`

Você pode forçar a conversão de um vetor de uma classe para outra com as funções `as.xxx` (sendo “xxx” a classe). Entretanto, nem sempre essa conversão faz sentido, e pode resultar em erros ou NA's. Por exemplo:

```
as.character(numero) # Vira texto
## [1] "546.9" "10"      "789"

as.numeric(logico) # TRUE -> 1, FALSE -> 0
## [1] 1 0 1

as.numeric(texto) # Não faz sentido
## Warning: NAs introduzidos por coerção
## [1] NA NA NA

as.numeric("1012312") # Faz sentido
## [1] 1e+06
```

## Estrutura e tamanho de um objeto

Para ver a **estrutura** de um objeto no R, use a função `str()`. Esta é uma função simples, mas talvez das mais úteis do R.

```
str(x3)
##  num [1:4] 10 2 3 20
str(numero)
##  num [1:3] 547 10 789
str(inteiro)
##  int [1:2] 1 98
str(complexo)
##  cplx [1:2] 0+20i 2+9i
str(texto)
##  chr [1:3] "aspas duplas" "aspas simples" "aspas 'dentro' do texto"
str(logico)
##  logi [1:3] TRUE FALSE TRUE
```

## Estrutura e tamanho de um objeto

Para obter o tamanho de um objeto, utilize a função `length()`. Comandos no R podem ser colocados na mesma linha se separados por ponto-e-vírgula (;).

```
length(x1);length(x3);length(x4)
## [1] 1
## [1] 4
## [1] 3

length(numero); length(inteiro); length(complexo)
## [1] 3
## [1] 2
## [1] 2

length(texto); length(logico)
## [1] 3
## [1] 3
```

## Nomes dos elementos de um objeto

Objetos podem ter elementos nomeados. Por exemplo, vamos nomear os elementos do vetor `numero`. A função `names()` serve tanto para consultar quanto para alterar os nomes de um objeto.

```
numero
## [1] 547 10 789

names(numero) <- c("numero1", "numero2", "numero3")
numero
## numero1 numero2 numero3
##      547      10      789

names(numero)
## [1] "numero1" "numero2" "numero3"
```

## Selecionando elementos de um vetor: índices

Você pode acessar elementos de um vetor por meio de colchetes (`[]`).

```
numero[1] # apenas primeiro elemento
## numero1
##      547
```

```
numero[c(1,2)] # elementos 1 e 2
## numero1 numero2
##      547      10
```

```
numero[c(1,3)] # elementos 1 e 3
## numero1 numero3
##      547      789
```

```
numero[c(3,1,2)] # troca de posição
## numero3 numero1 numero2
##      789      547      10
```

## Selecionando elementos de um vetor: índices negativos

Você pode usar índices negativos para omitir certos elementos:

```
numero[-1] # todos menos o primeiro
## numero2 numero3
##      10      789

numero[-c(1,2)] # todos menos 1 e 2
## numero3
##      789

numero[-c(1,3)] # todos menos 1 e 3
## numero2
##      10
```

## Selecionando elementos de um vetor: nomes

Isso também funciona com nomes, caso o vetor seja nomeado.

```
numero["numero1"]  
## numero1  
##      547  
  
numero["numero2"]  
## numero2  
##       10  
  
numero[c("numero1", "numero3")]  
## numero1 numero3  
##      547      789
```

## Selecionando elementos de um vetor: TRUE e FALSE

Por fim, vetores lógicos podem ser utilizados para selecionar elementos.

```
numero[c(TRUE, FALSE, FALSE)] # apenas primeiro elemento
## numero1
##      547

numero[c(FALSE, FALSE, TRUE)]  # apenas terceiro elemento
## numero3
##      789

numero[c(TRUE, FALSE, TRUE)] # elementos 1 e 3
## numero1 numero3
##      547      789
```

## Alterando elementos de um vetor

Você pode usar o assignment operator (<-) junto com colchetes ([ ]) para alterar elementos específicos do vetor.

```
numero
## numero1 numero2 numero3
##      547      10      789

numero[1] <- 100 # altera elemento 1 para 100
numero
## numero1 numero2 numero3
##      100      10      789

numero[2:3] <- c(12.3, -10) # altera elementos 2 e 3
numero
## numero1 numero2 numero3
##      100      12     -10
```

## Ordenando um vetor

A função `order()` retorna um vetor com as posições para que um objeto fique em ordem crescente.

```
order(numero) # indices
## [1] 3 2 1

numero[order(numero)] # ordena numero
## numero3 numero2 numero1
##      -10      12      100
```

A função `sort()` retorna o vetor ordenado.

```
sort(numero)
## numero3 numero2 numero1
##      -10      12      100
```

As duas funções tem o parâmetro `decreasing` que, quando `TRUE`, retornam o vetor em ordem decrescente.

## Criando sequências e repetições

Uma forma rápida e fácil de criar uma sequência de inteiros no R é utilizando dois pontos (:).

```
# Sequencia de 1 a 10
```

```
1:10
```

```
## [1] 1 2 3 4 5 6 7 8 9 10
```

```
# Sequencia de -1 a -10
```

```
-1:(-10)
```

```
## [1] -1 -2 -3 -4 -5 -6 -7 -8 -9 -10
```

## Criando sequências e repetições

Alternativamente, é possível criar sequências mais flexíveis com a função `seq()`:

```
seq(from = 1, to = 10, by = 3)
## [1] 1 4 7 10
```

E, para repetições, temos a função `rep()`:

```
rep(1, times = 10)
## [1] 1 1 1 1 1 1 1 1 1 1

rep(c(1,2), times = 5)
## [1] 1 2 1 2 1 2 1 2 1 2
```

# Vetorização, reciclagem, operadores matemáticos, relacionais e lógicos básicos

## Aritmética básica e vetorização

O R tem uma série de operadores de aritmética básica, entre eles:

| Operador   | Descrição                                 |
|------------|-------------------------------------------|
| $x + y$    | Soma (elemento a elemento)                |
| $x - y$    | Subtração (elemento a elemento)           |
| $x * y$    | Multipliação (elemento a elemento)        |
| $x / y$    | Divisão (elemento a elemento)             |
| $x ^ y$    | Exponenciação (elemento a elemento)       |
| $x \%/% y$ | Divisão por inteiro (elemento a elemento) |
| $x \% y$   | Resto da divisão (elemento a elemento)    |

### Vetorização!

Muitos operadores e funções do R são vetorizados, isto é, os cálculos são realizados elemento a elemento. Durante o curso esteja sempre atento a isso e tente explorar a vetorização do R para facilitar seus cálculos.

## Soma e Subtração

```
# soma  
1 + 20  
## [1] 21
```

```
# soma vetorizada (elemento a elemento)  
c(1,2,3) + c(4,5,6)  
## [1] 5 7 9
```

```
# subtração  
200 - 2  
## [1] 198
```

```
# subtração vetorizada (elemento a elemento)  
c(1,2,3) - c(4,5,6)  
## [1] -3 -3 -3
```

## Multiplicação e Divisão

```
# divisão
```

```
200 / 15
```

```
## [1] 13
```

```
# divisão vetorizada (elemento a elemento)
```

```
c(2,4,6) / c(1,2,3)
```

```
## [1] 2 2 2
```

```
# multiplicação
```

```
2*10
```

```
## [1] 20
```

```
# multiplicação vetorizada (elemento a elemento)
```

```
c(10,9,8) * c(1,2,3)
```

```
## [1] 10 18 24
```

## Exponenciação, Divisão por inteiro, Resto

```
# exponenciação
```

```
4^2
```

```
## [1] 16
```

```
# exponenciação vetorizada (elemento a elemento)
```

```
c(2,2,2) ^ c(1,2,3)
```

```
## [1] 2 4 8
```

```
# divisão por inteiro (desconsidera o resto)
```

```
7 %/% 3
```

```
## [1] 2
```

```
# divisão por inteiro vetorizada
```

```
c(7,7) %/% c(3,2)
```

```
## [1] 2 3
```

## Exponenciação, Divisão por inteiro, Resto

```
# resto da divisão  
7 %% 3  
## [1] 1
```

```
# resto da divisão vetorizada  
c(7,7) %% c(3,2)  
## [1] 1 1
```

## Reciclagem

O que acontece quando fazemos uma operação com vetores de tamanho diferente? Vejamos:

```
x <- c(1, 2, 3, 4)
x * 2
## [1] 2 4 6 8
```

Note que o R multiplicou todos os elementos de `x` por 2! Este processo é chamado de **reciclagem**, o R vai “expandindo” – ou “reciclando” – os valores do vetor menor até que este fique com a mesma quantidade de elementos que o vetor maior. Ou, mais especificamente, a operação acima é equivalente a:

```
x * c(2, 2, 2, 2)
## [1] 2 4 6 8
```

## Reciclagem

E o que ocorreria se o vetor menor tivesse dois elementos? A mesma coisa:

```
x <- c(1, 2, 3, 4)
x * c(2, 3)
## [1]  2  6  6 12

# equivalente
x * c(2, 3, 2, 3)
## [1]  2  6  6 12
```

E o que acontece com o exemplo a seguir? Agora note a mensagem de aviso (warning):

```
x * c(2, 3, 1)
## Warning in x * c(2, 3, 1): comprimento do objeto maior não é múltiplo do
## comprimento do objeto menor
## [1] 2 6 3 8
```

## Outras funções matemáticas

Além dos operadores básicos, há uma série de funções matemáticas comumente utilizadas, tais como:

| Função                    | Descrição          |
|---------------------------|--------------------|
| <code>abs(x)</code>       | Valor absoluto.    |
| <code>log(x)</code>       | Logaritmo natural. |
| <code>exp(x)</code>       | Exponencial.       |
| <code>sqrt(x)</code>      | Raiz quadrada.     |
| <code>factorial(x)</code> | Fatorial.          |

Todas essas funções **são vetorizadas**.

## Outras funções matemáticas

```
x <- c(1,2,-3, 4, -20.3) #criando o vetor x  
abs(x) # valor absoluto  
## [1] 1 2 3 4 20
```

```
# note que NaN's foram gerados, iremos falar disso mais a frente  
log(x)  
## Warning in log(x): NaNs produzidos  
## [1] 0.00 0.69 NaN 1.39 NaN
```

```
exp(x)  
## [1] 2.7e+00 7.4e+00 5.0e-02 5.5e+01 1.5e-09
```

```
sqrt(x) # NaN's gerados  
## Warning in sqrt(x): NaNs produzidos  
## [1] 1.0 1.4 NaN 2.0 NaN
```

## Outras funções matemáticas

```
# fatorial  
factorial(5) # 5*4*3*2*1  
## [1] 120
```

Há também diversas funções trigonométricas, como seno `sin()`, cosseno `cos()`, tangente `tan()` e outras.

```
sin(pi); cos(pi)  
## [1] 1.2e-16  
## [1] -1
```

Note que o resultado de  $\sin(\pi)$  não foi exatamente zero (mas é zero em termos práticos). Como qualquer outra linguagem, o R trabalha com números de ponto flutuante, então é preciso tomar algumas precauções. Falaremos mais disso adiante.

## Outras funções matemáticas

Funções que calculam estatísticas descritivas dos vetores:

| Função                                                                 | Descrição                                    |
|------------------------------------------------------------------------|----------------------------------------------|
| <code>sum(x)</code> e <code>cumsum(x)</code>                           | Soma e soma acumulada.                       |
| <code>prod(x)</code> e <code>cumprod(x)</code>                         | Produtório e produtório acumulado.           |
| <code>min(x)</code> , <code>cummin(x)</code> e <code>pmin(x, y)</code> | Mínimo, mínimo acumulado e mínimo par a par. |
| <code>max(x)</code> , <code>cummax(x)</code> e <code>pmax(x, y)</code> | Máximo, máximo acumulado e máximo par a par. |
| <code>mean(x)</code>                                                   | Média.                                       |
| <code>var(x)</code> e <code>sd(x)</code>                               | Variância e desvio-padrão.                   |
| <code>cov(x, y)</code> e <code>cor(x, y)</code>                        | Covariância e correlação.                    |
| <code>diff(x)</code>                                                   | Primeira diferença.                          |

## Outras funções matemáticas

Exemplos:

```
mean(x) # média  
## [1] -3.3
```

```
sum(x) # somatório  
## [1] -16
```

```
prod(x) # produto  
## [1] 487
```

```
cumsum(x) # somatório acumulado  
## [1] 1 3 0 4 -16
```

```
cumprod(x) # produto acumulado  
## [1] 1 2 -6 -24 487
```

## Outras funções matemáticas

Exemplos:

```
y <- 1:5

var(x) # variância  $\text{sum}((x - \text{mean}(x))^2) / (\text{length}(x) - 1)$ 
## [1] 97

sd(x) # desvio-padrão  $\text{sqrt}(\text{var}(x))$ 
## [1] 9.9

median(x) # mediana
## [1] 1

cov(x, y) # covariância  $\text{sum}((x - \text{mean}(x)) * (y - \text{mean}(y))) / (\text{length}(x) - 1)$ 
## [1] -10

cor(x, y) # correlação de x e y  $\text{cov}(x, y) / (\text{sd}(x) * \text{sd}(y))$ 
## [1] -0.65
```

## Outras funções matemáticas

Exemplos:

```
min(x) # mínimo
## [1] -20

max(x) # máximo
## [1] 4

cummin(x) # mínimo "acumulado"
## [1] 1 1 -3 -3 -20

cummax(x) # máximo "acumulado"
## [1] 1 2 2 4 4

diff(x) # diferença
## [1] 1 -5 7 -24
```

## Operadores Relacionais

Vejamos agora alguns operadores relacionais. Estes operadores são importantes para determinar a relação entre dois vetores e, como seu resultado é um vetor lógico, são muito utilizados para fazer subset. Da mesma forma que os operadores matemáticos, operadores relacionais também são vetorizados.

| Operador               | Descrição                   |
|------------------------|-----------------------------|
| <code>x == y</code>    | x é igual a y? Faz coerção. |
| <code>x != y</code>    | x é diferente de y?         |
| <code>x &gt; y</code>  | x é maior do que y?         |
| <code>x &gt;= y</code> | x é maior ou igual a y?     |
| <code>x &lt; y</code>  | x é menor do que y?         |
| <code>x &lt;= y</code> | x é menor ou igual a y?     |

## Operadores Relacionais

Para comparar se dois objetos são iguais, use ==.

```
x <- 10
y <- 20
x == y
## [1] FALSE

x <- c(10, 20, 30)
y <- c(10, 10, 30)
x == y
## [1] TRUE FALSE TRUE

x[x == y] # pega elementos de x que são iguais a y
## [1] 10 30
```

## Operadores Relacionais

Se você comparar objetos de classes diferentes (um texto com um número, por exemplo), sempre que possível, o operador `==` irá converter um dos objetos que está sendo comparado para tentar realizar a comparação.

```
x <- c(10, 20)
y <- c("10", "20")
x == y # converte x para texto e compara textualmente.
## [1] TRUE TRUE
```

É preciso tomar cuidado com coerções, pois isso pode gerar resultados inesperados:

```
20 > "100" # -> correto do ponto de vista textual (dicionário)
## [1] TRUE
```

## Operadores Relacionais

Caso você queira verificar se dois vetores são exatamente idênticos, você não deve utilizar `==` e sim `identical()`.

```
identical(x, y)
## [1] FALSE
```

Voltemos, agora, ao caso do valor calculado para  $\sin(\pi)$ . Lembra que o valor não foi exatamente zero? Isso ocorre porque computadores trabalham com números de ponto flutuante e não têm precisão infinita.

```
identical(sin(pi), 0)
## [1] FALSE

identical(0.1, 0.3 - 0.2)
## [1] FALSE
```

## Operadores Relacionais

Assim, sempre que for fazer comparações de números resultantes de cálculos, é preciso considerar uma tolerância de erro. Uma função que faz isso é a `all.equal()` que leva em conta uma tolerância de erro de ponto flutuante.

```
sin(pi) == 0 # falso, incorreto.  
## [1] FALSE  
  
all.equal(sin(pi), 0) # verdadeiro, correto  
## [1] TRUE
```

## Operadores Relacionais

Ou você pode testar se a diferença absoluta entre um número e outro é irrelevante (do ponto de vista numérico):

```
abs(sin(pi) - 0) < 1e-12  
## [1] TRUE
```

## Operadores Relacionais

Além do operador `==`, como vimos, também temos os operadores: maior `>`, maior ou igual `>=`, menor `<`, menor ou igual `<=` e diferente `!=`. Note, novamente, que os resultados de todas essas operações são objetos do R, mais especificamente, vetores lógicos e podem ser utilizados em operações subsequentes:

```
x <- c(1,2,3,4,5)
y <- c("1","3","2","4","4");

# guarda resultado da comparação em vetor_logico
vetor_logico <- (x >= y)
vetor_logico
## [1] TRUE FALSE TRUE TRUE TRUE

# usa vetor logico para fazer subset de x
x[vetor_logico]
## [1] 1 3 4 5
```

## Operadores Relacionais

Um fato bastante útil de vetores lógicos é o de que eles, quando convertidos para numéricos, são transformados em vetores de 0's e 1's. Assim, por exemplo, no caso anterior, se quisermos saber quantos  $x$  são maiores ou iguais a  $y$ , basta somar os 1's do `vetor_logico` ou, se quisermos saber a proporção de  $x$  que são maiores ou iguais a  $y$ , basta tirarmos a média do `vetor_logico`.

```
as.numeric(vetor_logico)
## [1] 1 0 1 1 1
```

```
sum(vetor_logico)
## [1] 4
```

```
mean(vetor_logico)
## [1] 0.8
```

## Operadores lógicos

Podemos operar também com os próprios valores lógicos. O operador `!` nega o valor lógico ao qual é aplicado, isto é, `!FALSE` vira `TRUE` e `!TRUE` vira `FALSE`. No caso anterior, se quisermos saber quantos `x` são menores do que `y`, não precisamos comparar novamente, basta negar o resultado do `vetor_logico` e somar.

```
!vetor_logico
## [1] FALSE  TRUE FALSE FALSE FALSE

sum(!vetor_logico)
## [1] 1

identical(!vetor_logico, (x < y)) # são idênticos
## [1] TRUE
```

## Operadores lógicos

Operadores lógicos E(AND) & e OU(OR) |.

```
c(TRUE, FALSE) & c(TRUE, TRUE)
## [1] TRUE FALSE
```

```
c(TRUE, FALSE) | c(TRUE, TRUE)
## [1] TRUE TRUE
```

## Funções sobre vetores lógicos

Outros comandos que podem ser bastante úteis são o `all()` e o `any()`. O primeiro retorna TRUE se todos os elementos forem TRUE. Já o segundo retorna TRUE se ao menos um elemento for TRUE.

```
set.seed(11) # semente da simulação
x <- rnorm(1000, 5, 2) # simula 1000 obs normal(5, 2)

any(x > 10) # há algum x maior do que 10
## [1] TRUE

all(x > -20 & x < 20) # todos os x estão entre -1 e 20?
## [1] TRUE
```

## Funções sobre vetores lógicos

A função `which()` retorna a posição dos elementos que são TRUE.

```
which(c(TRUE, FALSE, TRUE))  
## [1] 1 3  
  
# quais as posições dos elementos de x  
# que são maiores do que 10?  
which(x > 10)  
## [1] 196 273 523 558 929
```

## Funções de conjuntos

| Função                       | Descrição                                      |
|------------------------------|------------------------------------------------|
| <code>setdiff(x, y)</code>   | Conjunto dos elementos de x que não estão em y |
| <code>intersect(x, y)</code> | Conjunto dos elementos de x que estão em y     |
| <code>union(x, y)</code>     | Conjunto união de x e y                        |
| <code>x %in% y</code>        | Quais elementos de x estão em y?               |

## Funções de conjuntos

Exemplos:

```
x <- 1:5
y <- c(1:3, 6:10)

setdiff(x, y)
## [1] 4 5

intersect(x, y)
## [1] 1 2 3

union(x, y)
## [1] 1 2 3 4 5 6 7 8 9 10

x %in% y
## [1] TRUE TRUE TRUE FALSE FALSE
```

## NA, NaN, Infinito

Em muitos casos, o resultado de uma operação é infinito ou não determinado. Outras vezes, há valores ausentes em sua base de dados. O R tem objetos especiais para lidar com esses tipos de situação. Por exemplo, quando tiramos o log de um número negativo, obtemos como resultado o valor NaN (Not an Number). Outros valores de interesse são Inf (Infinito) e NA (Not Available).

```
log(-1) #NaN
## Warning in log(-1): NaNs produzidos
## [1] NaN

x <- NaN # atribuindo o valor NaN a x

x + 1 # somar 1 a algo indeterminado é indeterminado
## [1] NaN

NaN == NaN # Não faça isso... comparação lógica, note que deu NA
## [1] NA
```

## NA, NaN, Infinito

Infinito:

```
2/0 # infinito
## [1] Inf

x <- 2/0

x - Inf # infinito - infinito não é um número (NaN)
## [1] NaN

1/x # zero
## [1] 0

is.infinite(x)
## [1] TRUE

is.finite(x)
## [1] FALSE
```

## NA, NaN, Infinito

O NA representa valores ausentes e é bastante utilizado quando se lida com bases de dados. Muitas funções têm parâmetros que indicam como ela deve lidar com valores ausentes.

```
x <- c(1, 2, NA, 3)
x + 1
## [1] 2 3 NA 4

x + Inf
## [1] Inf Inf NA Inf

sum(x)
## [1] NA

sum(x, na.rm = TRUE) #na.rm, opção para remover (rm) os NA (na)
## [1] 6
```

## NA, NaN, Infinito

Para testar se um elemento é NA, use a função `is.na()`:

```
x == NA # Nunca faça isso!!!
## [1] NA NA NA NA

is.na(x) # Maneira correta
## [1] FALSE FALSE TRUE FALSE
```

Matrizes: vetores com dimensões  
Data Frames: seu banco de dados no R  
Listas: juntando várias coisas diferentes em um só objeto.

## Matrizes: vetores com dimensões

## Criando matrizes: função `matrix()`

É possível criar matrizes no R com a função `matrix()`.

```
matrix(data = dados, ncol = numero_de_colunas, nrow = numero_de_linhas)
```

- No argumento `data` passamos o vetor que desejamos transformar em matriz;
- `ncol` especifica o número de colunas da matriz; e,
- `nrow` especifica o número de linhas da matriz.

## Criando matrizes: função `matrix()`

Criemos nossa primeira matriz, com 10 elementos, sendo cinco linhas e duas colunas:

```
minha_matriz <- matrix(data = 1:10, ncol = 2, nrow = 5)
minha_matriz
##      [,1] [,2]
## [1,]    1    6
## [2,]    2    7
## [3,]    3    8
## [4,]    4    9
## [5,]    5   10
```

Perceba que os elementos foram preenchidos por coluna. Isto é, os números de 1 a 5 ficaram na primeira coluna e os números de 6 a 10 na segunda.

## Criando matrizes: função `matrix()`

Para que a função preencha a matriz por linhas, basta colocar como argumento `byrow = TRUE`:

```
minha_matriz_2 <- matrix(data = 1:10, ncol = 2, nrow = 5, byrow = TRUE)
minha_matriz_2
##      [,1] [,2]
## [1,]    1    2
## [2,]    3    4
## [3,]    5    6
## [4,]    7    8
## [5,]    9   10
```

A matriz agora foi preenchida por linhas: temos os números 1 e 2 na primeira linha, 3 e 4 na segunda linha e assim por diante.

## Omitindo ncol ou nrow

Você não precisa especificar os dois termos `ncol` e `nrow`, basta especificar um deles que o outro é automaticamente inferido pelo R.

```
matrix(data = 1:10, ncol = 5)
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    3    5    7    9
## [2,]    2    4    6    8   10
```

```
matrix(data = 1:10, nrow = 2)
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    3    5    7    9
## [2,]    2    4    6    8   10
```

## Estrutura e atributos de uma matriz

A função `str()` fornece algumas informações básicas sobre a estrutura da matriz:

```
str(minha_matriz)
##  int [1:5, 1:2] 1 2 3 4 5 6 7 8 9 10
```

Na descrição `int [1:5, 1:2]` vemos que o objeto `minha_matriz` é composto de inteiros (`int`) e tem duas dimensões, mais especificamente cinco linhas (`[1:5, ...]`) e duas colunas (`[..., 1:2]`).

## Estrutura e atributos de uma matriz

```
# número de linhas da matriz
nrow(minha_matriz)
## [1] 5

# número de colunas da matriz
ncol(minha_matriz)
## [1] 2

# dimensões da matriz (# linhas, # colunas)
dim(minha_matriz)
## [1] 5 2

# tamanho da matriz (quantos elementos no total)
length(minha_matriz)
## [1] 10
```

## Nomes das linhas e colunas

As linhas e colunas de uma matriz podem ser nomeadas. Para tanto você pode utilizar `rownames()` e `colnames()`:

```
rownames(minha_matriz) <- letters[1:5]
rownames(minha_matriz)
## [1] "a" "b" "c" "d" "e"

colnames(minha_matriz) <- LETTERS[1:2]
colnames(minha_matriz)
## [1] "A" "B"

str(minha_matriz)
## int [1:5, 1:2] 1 2 3 4 5 6 7 8 9 10
## - attr(*, "dimnames")=List of 2
## ..$ : chr [1:5] "a" "b" "c" "d" ...
## ..$ : chr [1:2] "A" "B"
```

## Matrizes são vetores com um atributo a mais: dimensão

Outra forma bem simples de criar uma matriz é atribuir dimensões a um vetor. Por exemplo, podemos criar um objeto idêntico à `minha_matriz` da seguinte forma:

```
# cria vetor com números de 1 a 10
m <- 1:10

# atribui dimensões ao vetor (5 linhas e 2 colunas)
dim(m) <- c(5, 2)
m
##      [,1] [,2]
## [1,]    1    6
## [2,]    2    7
## [3,]    3    8
## [4,]    4    9
## [5,]    5   10
```

## Operações com matrizes

| Operador                                                                                                  | Descrição                                                                                     |
|-----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <code>==</code> <code>!=</code> <code>&gt;</code> <code>&gt;=</code> <code>&lt;</code> <code>&lt;=</code> | Operadores relacionais elemento a elemento                                                    |
| <code>+</code> <code>-</code> <code>/</code> <code>*</code>                                               | Soma, subtração, divisão e multiplicação elemento a elemento                                  |
| <code>%*%</code>                                                                                          | Multiplicação de matriz                                                                       |
| <code>%x%</code>                                                                                          | Produto de Kronecker                                                                          |
| <code>t()</code>                                                                                          | Transposição de matriz                                                                        |
| <code>solve()</code>                                                                                      | Inversão de matriz (ver também <code>qr()</code> , <code>svd()</code> , <code>chol()</code> ) |
| <code>det()</code>                                                                                        | Determinante de uma matriz                                                                    |
| <code>diag()</code>                                                                                       | Diagonal de uma matriz                                                                        |

## Operações elemento a elemento: soma, multiplicação e divisão

Vamos criar uma matriz mais simples para testar essas operações:

```
z <- 1:4  
dim(z) <- c(2, 2)  
str(z)  
## int [1:2, 1:2] 1 2 3 4
```

Primeiramente a soma (elemento a elemento):

```
# soma elemento a elemento  
z + z  
##      [,1] [,2]  
## [1,]    2    6  
## [2,]    4    8
```

## Operações elemento a elemento: soma, multiplicação e divisão

A soma não precisa ser de matriz com outra matriz. É possível somar uma matriz com um vetor. Neste caso, a soma é feita seguindo a ordem por coluna:

```
z + c(1, 2, 3, 4)
##      [,1] [,2]
## [1,]    2    6
## [2,]    4    8
```

## Operações elemento a elemento: soma, multiplicação e divisão

Quando os elementos não são do mesmo tamanho, o R tenta fazer reciclagem (a mesma que vimos na seção de vetores).

```
z + 10
##      [,1] [,2]
## [1,]   11  13
## [2,]   12  14
```

Repare que o R somou cada elemento de z com o número 10. E se somássemos um vetor com dois elementos?

```
z + c(10, 20)
##      [,1] [,2]
## [1,]   11  13
## [2,]   22  24
```

Note que o R não reclamou da operação! Ele reciclou o vetor c(10, 20) para as duas colunas de z.

## Operações elemento a elemento: soma, multiplicação e divisão

E se o vetor somado fosse de três elementos?

```
z + c(10, 20, 30)
## Warning in z + c(10, 20, 30): comprimento do objeto maior não é múltiplo do
## comprimento do objeto menor
##      [,1] [,2]
## [1,]  11  33
## [2,]  22  14
```

O R faz a operação mas te dá um aviso, pois o tamanho do vetor não é um múltiplo do tamanho da matriz. A reciclagem é uma característica bastante útil do R, mas é preciso tomar cuidado.

## Operações elemento a elemento: soma, multiplicação e divisão

O mesmo que falamos para soma vale para as demais operações elemento a elemento:

```
z_mais_z <- z + z
z_mais_z
##      [,1] [,2]
## [1,]    2    6
## [2,]    4    8

dois_z <- 2*z
z_mais_z == dois_z
##      [,1] [,2]
## [1,] TRUE TRUE
## [2,] TRUE TRUE
```

## Operações matriciais

Multiplicação matricial:

```
zz <- z %*% z
zz
##      [,1] [,2]
## [1,]    7  15
## [2,]   10  22
```

Transposição:

```
tz <- t(zz)
tz
##      [,1] [,2]
## [1,]    7  10
## [2,]   15  22
```

## Operações matriciais

Inversão:

```
inv_zz <- solve(zz)
inv_zz
##      [,1] [,2]
## [1,]  5.5 -3.7
## [2,] -2.5  1.7
```

Determinante:

```
det(zz)
## [1] 4
```

## Operações matriciais

Produto de kronecker:

```
z %x% z
##      [,1] [,2] [,3] [,4]
## [1,]    1    3    3    9
## [2,]    2    4    6   12
## [3,]    2    6    4   12
## [4,]    4    8    8   16
```

Diagonal:

```
diag(z)
## [1] 1 4
```

## Exemplo: regressão linear

Vamos simular e estimar os parâmetros de uma regressão linear usando operações matriciais:

```
# parametros
n_vars <- 4 # numero de variaveis
n_obs <- 100 # numero de observações

# Simulando y ~ Xb + e
X <- matrix(rnorm(n_obs*n_vars), ncol = n_vars) # vetor X
betas <- c(1, 2, 3, 4) # betas verdadeiros
erro <- rnorm(n_obs) # termo de erro normal(0,1)
y <- X %*% betas + erro # vetor y
```

## Exemplo: regressão linear

Estimando por mínimos quadrados:

```
# estimando os betas:  $(X'X)^{-1} X'y$ 
betas_estimados <- solve(t(X) %*% X) %*% t(X) %*% y
betas_estimados[,1]
## [1] 0.95 1.90 2.96 4.17

# Comparando com a estimativa da função nativa do R
modelo <- lm(y ~ X - 1)
coef(modelo)
##      X1      X2      X3      X4
## 0.95 1.90 2.96 4.17
```

## Nomes das linhas e colunas

Vamos criar uma matriz de exemplo mais realista:

```
# criando matriz de vendas para exemplo  
# quantidade de vendedores  
n_vendedores <- 3  
  
# quantidade de dias  
n_dias <- 5  
  
# valores de venda (aleatórios)  
# média de R$1000,00 por dia  
# desvio padrão de R$200,00  
set.seed(192)  
media <- 1000  
desvio <- 200  
valores <- rnorm(n_dias*n_vendedores, media, desvio)
```

## Nomes das linhas e colunas

```
# cria matriz
vendas <- matrix(valores, nrow = n_dias, ncol = n_vendedores)

# nomes para linhas
rownames(vendas) <- c("segunda", "terça", "quarta", "quinta", "sexta")

# nomes para colunas
colnames(vendas) <- c("João", "Maria", "Ana")
```

## Nomes das linhas e colunas

Vamos ver como ficou nossa matriz:

```
vendas
##      João Maria  Ana
## segunda 1201   927 1043
## terça   1027   920  833
## quarta  1096  1020  944
## quinta   915   952 1018
## sexta   787  1039 1293
```

## Selecionando elementos de uma matriz

Tudo o que vimos para selecionar elementos de um vetor vale para matrizes. A principal diferença é que agora você vai estabelecer um índice para linha e outro para a coluna do elemento que você quer:

```
matriz[indice_de_linha, indice_de_coluna]
```

## Selecionando elementos de uma matriz

Selecionando linhas (por nome ou posição):

```
vendas["segunda", ] # seleciona linha de nome "segunda"  
## João Maria Ana  
## 1201 927 1043
```

```
vendas[1, ] # seleciona primeira linha  
## João Maria Ana  
## 1201 927 1043
```

```
vendas[-c(1,2), ] # seleciona tudo exceto linhas 1 e 2  
## João Maria Ana  
## quarta 1096 1020 944  
## quinta 915 952 1018  
## sexta 787 1039 1293
```

## Selecionando elementos de uma matriz

Selecionando colunas (por nome ou posição):

```
vendas[ , "Ana"] # seleciona colunas de nome "Ana"
## segunda  terça  quarta  quinta  sexta
##    1043    833    944    1018    1293
```

```
vendas[ , 3] # seleciona terceira coluna
## segunda  terça  quarta  quinta  sexta
##    1043    833    944    1018    1293
```

```
vendas[ , -3] # seleciona tudo exceto terceira coluna
##           João Maria
## segunda 1201    927
## terça   1027    920
## quarta  1096   1020
## quinta   915    952
## sexta   787   1039
```

## Selecionando elementos de uma matriz

Selecionando linhas e colunas (por nome ou posição). Resultados omitidos.

```
# linhas 1, 3 e 5 - colunas João e Ana  
vendas[c(1, 3, 5), c("João", "Ana")]
```

```
# linhas segunda, quarta e sexta - colunas 1 e 3  
vendas[c("segunda", "quarta", "sexta"), c(1, 3)]
```

```
# tudo menos linhas 1, 3 e 5 e menos colunas 2 e 3  
vendas[-c(1,3,5), -c(2,3)]
```

## Cuidado: simplificação de matrizes e drop = FALSE

Uma coisa ao ter cuidado ao fazer subsets com matrizes é que, quando a seleção tem apenas uma linha ou uma coluna, o R simplifica o resultado para um vetor. Na maior parte das vezes isso é bastante conveniente, pois é de fato o que queremos, mas em alguns casos pode ser desejável manter a estrutura de matriz no resultado. Para tanto, basta colocar como argumento `drop = FALSE`.

```
vendas_segunda <- vendas[1, , drop = FALSE]
vendas_segunda
##           João Maria   Ana
## segunda 1201    927 1043

# resultado é uma matriz
dim(vendas_segunda)
## [1] 1 3
```

## Alterando valores de uma matriz

É possível combinar a seleção de elementos com o operador `<-` para realizar alterações em uma matriz:

```
vendas[c("quarta", "quinta"), "Ana"]  
## quarta quinta  
##      944    1018  
  
vendas[c("quarta", "quinta"), "Ana"] <- c(1500, 2000)  
  
vendas[c("quarta", "quinta"), "Ana"]  
## quarta quinta  
##    1500    2000
```

## Adicionando colunas com cbind()

```
set.seed(145)
José <- rnorm(nrow(vendas), media, desvio)
vendas <- cbind(vendas, José)
vendas
```

| ## |         | João | Maria | Ana  | José |
|----|---------|------|-------|------|------|
| ## | segunda | 1201 | 927   | 1043 | 1137 |
| ## | terça   | 1027 | 920   | 833  | 1213 |
| ## | quarta  | 1096 | 1020  | 1500 | 1107 |
| ## | quinta  | 915  | 952   | 2000 | 1381 |
| ## | sexta   | 787  | 1039  | 1293 | 1213 |

## Adicionando linhas com `rbind()`

```
set.seed(90)
sábado <- rnorm(ncol(vendas), media, desvio)
vendas <- rbind(vendas, sábado)
vendas
```

| ## |         | João | Maria | Ana  | José |
|----|---------|------|-------|------|------|
| ## | segunda | 1201 | 927   | 1043 | 1137 |
| ## | terça   | 1027 | 920   | 833  | 1213 |
| ## | quarta  | 1096 | 1020  | 1500 | 1107 |
| ## | quinta  | 915  | 952   | 2000 | 1381 |
| ## | sexta   | 787  | 1039  | 1293 | 1213 |
| ## | sábado  | 1015 | 970   | 823  | 856  |

## Removendo linhas e colunas: basta não selecionar!

Para remover uma linha, basta sobrescrever a matriz original **não selecionando** a linha que você deseja excluir.

```
# Remove a sexta linha  
vendas <- vendas[-6, ]
```

A mesma lógica para remover uma coluna:

```
# Remove a quarta coluna  
vendas <- vendas[ , -4]
```

## Aplicando funções nas linhas e colunas: Summary

A função `summary()` fornece várias estatísticas descritivas por coluna:

```
summary(vendas)
```

| ##          | João  | Maria        | Ana          |
|-------------|-------|--------------|--------------|
| ## Min.     | : 787 | Min. : 920   | Min. : 833   |
| ## 1st Qu.: | 915   | 1st Qu.: 927 | 1st Qu.:1043 |
| ## Median   | :1027 | Median : 952 | Median :1293 |
| ## Mean     | :1005 | Mean : 972   | Mean :1334   |
| ## 3rd Qu.: | 1096  | 3rd Qu.:1020 | 3rd Qu.:1500 |
| ## Max.     | :1201 | Max. :1039   | Max. :2000   |

## Aplicando funções nas linhas e colunas: Summary

Se você quise usar `summary()` nas linhas, basta transpor a matriz:

```
summary(t(vendas))
```

| ## | segunda      | terça        | quarta       | quinta       | sexta      |
|----|--------------|--------------|--------------|--------------|------------|
| ## | Min. : 927   | Min. : 833   | Min. :1020   | Min. : 915   | Min. : 78  |
| ## | 1st Qu.: 985 | 1st Qu.: 877 | 1st Qu.:1058 | 1st Qu.: 934 | 1st Qu.: 9 |
| ## | Median :1043 | Median : 920 | Median :1096 | Median : 952 | Median :10 |
| ## | Mean :1057   | Mean : 927   | Mean :1205   | Mean :1289   | Mean :10   |
| ## | 3rd Qu.:1122 | 3rd Qu.: 973 | 3rd Qu.:1298 | 3rd Qu.:1476 | 3rd Qu.:11 |
| ## | Max. :1201   | Max. :1027   | Max. :1500   | Max. :2000   | Max. :12   |

## Aplicando funções nas linhas e colunas: rowSums e rowMeans

O R também já fornece duas funções para somas e médias por linhas:

```
# Total de vendas por dia
```

```
rowSums(vendas)
```

```
## segunda  terça  quarta  quinta  sexta  
##      3171    2780    3616    3867    3119
```

```
# Média das vendas por dia
```

```
rowMeans(vendas)
```

```
## segunda  terça  quarta  quinta  sexta  
##      1057    927    1205    1289    1040
```

## Aplicando funções nas linhas e colunas: colSums e colMeans

E somas e médias por colunas:

```
# Total de vendas por vendedor
colSums(vendas)
## João Maria Ana
## 5025 4858 6670

# Média de vendas por vendedor
colMeans(vendas)
## João Maria Ana
## 1005 972 1334
```

Mas e se eu quiser aplicar uma função diferente de `sum()` ou `mean()`?

Matrizes: vetores com dimensões  
Data Frames: seu banco de dados no R  
Listas: juntando várias coisas diferentes em um só objeto.

## Data Frames: seu banco de dados no R

Matrizes: vetores com dimensões  
Data Frames: seu banco de dados no R  
Listas: juntando várias coisas diferentes em um só objeto.

## Por que um `data.frame`?

Até agora temos utilizado apenas dados de uma mesma classe, armazenados ou em um vetor ou em uma matriz. Mas uma base de dados, em geral, é feita de dados de diversas classes diferentes: no exemplo anterior, por exemplo, podemos querer ter uma coluna com os nomes dos funcionários, outra com o sexo dos funcionários, outra com valores... como guardar essas informações?

Matrizes: vetores com dimensões  
Data Frames: seu banco de dados no R  
Listas: juntando várias coisas diferentes em um só objeto.

## Por que um `data.frame`?

A solução para isso é o `data.frame`. O `data.frame` é talvez o formato de dados mais importante do R. No `data.frame` cada coluna representa uma variável e cada linha uma observação. Essa é a estrutura ideal para quando você tem várias variáveis de classes diferentes em um banco de dados.

## Criando um data.frame: data.frame()

É possível criar um data.frame diretamente com a função data.frame():

```
funcionarios <- data.frame(nome = c("João", "Maria", "José"),  
                             sexo = c("M", "F", "M"),  
                             salario = c(1000, 1200, 1300),  
                             stringsAsFactors = FALSE)
```

```
funcionarios  
##      nome sexo  salario  
## 1  João    M     1000  
## 2 Maria    F     1200  
## 3  José    M     1300
```

***Discutiremos a opção stringsAsFactors = FALSE mais a frente.***

## Criando um data.frame: `data.frame()`

Vejamos a estrutura do `data.frame`. Note que cada coluna tem sua própria classe.

```
str(funcionarios)
## 'data.frame':    3 obs. of  3 variables:
##  $ nome      : chr  "João" "Maria" "José"
##  $ sexo      : chr  "M" "F" "M"
##  $ salario: num  1000 1200 1300
```

## Criando um data.frame: data.frame()

O que ocorreria com o data.frame funcionarios se o transformássemos em uma matriz? Vejamos:

```
as.matrix(funcionarios)
##      nome      sexo salario
## [1,] "João"    "M"    "1000"
## [2,] "Maria"   "F"    "1200"
## [3,] "José"    "M"    "1300"
```

Perceba que todas as variáveis viraram character! Exatamente por isso precisamos de um data.frame neste caso.

## Criando um data.frame: `as.data.frame()`

Também é possível criar um `data.frame` com a função `as.data.frame()`. Voltando ao exemplo das vendas:

```
df.vendas <- as.data.frame(t(vendas))
df.vendas
##           segunda terça quarta quinta sexta
## João         1201  1027   1096    915   787
## Maria         927   920   1020    952  1039
## Ana          1043   833   1500   2000  1293

str(df.vendas)
## 'data.frame':   3 obs. of  5 variables:
## $ segunda: num  1201 927 1043
## $ terça : num  1027 920 833
## $ quarta : num  1096 1020 1500
## $ quinta : num  915 952 2000
## $ sexta : num  787 1039 1293
```

Matrizes: vetores com dimensões  
Data Frames: seu banco de dados no R  
Listas: juntando várias coisas diferentes em um só objeto.

## Manipulando `data.frames` como matrizes

Ok, temos mais um objeto do R, o `data.frame` ... vou ter que reaprender tudo novamente? Não!

Você pode manipular `data.frames` como se fossem matrizes!

Praticamente tudo o que vimos para selecionar e modificar elementos em matrizes funciona no `data.frame`.

## Manipulando data.frames como matrizes

Selecionando linhas e colunas:

```
# tudo menos linha 1
funcionarios[-1, ]
##      nome sexo salario
## 2 Maria    F    1200
## 3  José    M    1300
```

```
# seleciona primeira linha e primeira coluna (vetor)
funcionarios[1, 1]
## [1] "João"
```

```
# seleciona primeira linha e primeira coluna (data.frame)
funcionarios[1, 1, drop = FALSE]
##      nome
## 1 João
```

## Manipulando data.frames como matrizes

Alterando valores do data.frame como se fosse uma matriz.

```
# aumento de salario para o João  
funcionarios[1, "salario"] <- 1100
```

```
funcionarios  
##      nome sexo salario  
## 1  João    M    1100  
## 2 Maria    F    1200  
## 3  José    M    1300
```

## Nomes de linhas e colunas

O data.frame sempre terá rownames e colnames.

```
rownames(funcionarios)
## [1] "1" "2" "3"

colnames(funcionarios)
## [1] "nome"      "sexo"      "salario"
```

Detalhe: o names trata-se de nomes das colunas do data.frame. **Os elementos do data.frame são seus vetores coluna.**

```
names(funcionarios)
## [1] "nome"      "sexo"      "salario"
```

## Extra do data.frame: selecionando e modificando com \$ e [[ ]]

Outras formas alternativas de selecionar colunas em um data.frame são o \$ e o [[ ]]:

```
# Seleciona coluna nome
funcionarios$nome
## [1] "João" "Maria" "José"

funcionarios[["nome"]]
## [1] "João" "Maria" "José"

# Seleciona coluna salario
funcionarios$salario
## [1] 1100 1200 1300

funcionarios[["salario"]]
## [1] 1100 1200 1300
```

Tanto o \$ quanto o [[ ]] **sempre** retornam um vetor como resultado.

## Extra do data.frame: selecionando e modificando com \$ e [[ ]]

Também é possível alterar a coluna combinando \$ ou [[ ]] com <-:

```
# outro aumento para o João
funcionarios$salario[1] <- 1150

# equivalente
funcionarios[["salario"]][1] <- 1150
funcionarios
##      nome sexo salario
## 1  João    M    1150
## 2 Maria    F    1200
## 3  José    M    1300
```

## Extra do data.frame: retornando sempre um data.frame com [ ]

Se você quiser garantir que o resultado da seleção será sempre um data.frame use `drop = FALSE` ou selecione sem a vírgula:

```
# Retorna data.frame
funcionarios[ , "salario", drop = FALSE]
##   salario
## 1    1150
## 2    1200
## 3    1300

# Retorna data.frame
funcionarios["salario"]
##   salario
## 1    1150
## 2    1200
## 3    1300
```

## Tabela resumo: selecionando uma coluna em um data.frame

Resumindo as formas de seleção de uma coluna de um data.frame.

| Operador                             | Descrição                                    |
|--------------------------------------|----------------------------------------------|
| <code>df[, "x"]</code>               | Retorna <b>vetor</b> x do data.frame df.     |
| <code>df\$x</code>                   | Retorna <b>vetor</b> x do data.frame df.     |
| <code>df[["x"]]</code>               | Retorna <b>vetor</b> x do data.frame df.     |
| <code>df[, "x", drop = FALSE]</code> | Retorna um <b>data.frame</b> com a coluna x. |
| <code>df["x"]</code>                 | Retorna um <b>data.frame</b> com a coluna x. |

## Criando colunas novas

Com \$:

```
funcionarios$escolaridade <- c("Ensino Médio", "Graduação", "Mestrado")
```

Com [, ]:

```
funcionarios[, "experiencia"] <- c(10, 12, 15)
```

Com [[ ]]:

```
funcionarios[["avaliacao_anual"]] <- c(7, 9, 10)
```

Com cbind():

```
funcionarios <- cbind(funcionarios,  
  prim_emprego = c("sim", "nao", "nao"),  
  stringsAsFactors = FALSE)
```

## Criando colunas novas

Vejamos como ficou nosso data.frame com as novas colunas:

```
funcionarios
##      nome sexo  salario escolaridade experiencia avaliacao_anual prim_emprego
## 1  João    M   1150  Ensino Médio          10              7          sim
## 2  Maria    F   1200   Graduação          12              9          nao
## 3  José     M   1300   Mestrado           15             10          nao
```

## Removendo colunas

Atribuindo NULL:

```
# deleta coluna prim_emprego  
funcionarios$prim_emprego <- NULL
```

Ou selecionando todas colunas menos as que você não quer:

```
# deleta colunas 4 e 6  
funcionarios <- funcionarios[, c(-4, -6)]
```

## Adicionando linhas

Uma forma simples de adicionar linhas é atribuir a nova linha com `<-`. Mas cuidado! O que irá acontecer com o `data.frame` com o código abaixo?

```
# CUIDADO!  
funcionarios[4, ] <- c("Ana", "F", 2000, 15)
```

Note que nosso `data.frame` inteiro se transformou em texto! Você sabe explicar por que isso aconteceu?

```
str(funcionarios)  
## 'data.frame':    4 obs. of  4 variables:  
## $ nome          : chr  "João" "Maria" "José" "Ana"  
## $ sexo          : chr  "M" "F" "M" "F"  
## $ salario       : chr  "1150" "1200" "1300" "2000"  
## $ experiencia: chr  "10" "12" "15" "15"
```

## Adicionando linhas

Antes de prosseguir, transformemos as colunas salario e experiencia em números novamente:

```
funcionarios$salario <- as.numeric(funcionarios$salario)

funcionarios$experiencia <- as.numeric(funcionarios$experiencia)
```

## Adicionando linhas

Se os elementos forem de classe diferente, use a função `data.frame` para evitar coerção:

```
funcionarios[4, ] <- data.frame(nome = "Ana", sexo = "F",  
                                salario = 2000, experiencia = 15,  
                                stringsAsFactors = FALSE)
```

Também é possível adicionar linhas com `rbind()`:

```
rbind(funcionarios,  
      data.frame(nome = "Ana", sexo = "F",  
                  salario = 2000, experiencia = 15,  
                  stringsAsFactors = FALSE))
```

**Atenção! Não fique aumentando um `data.frame` de tamanho adicionando linhas ou colunas. Sempre que possível pré-aloque espaço!**

## Removendo linhas

Para remover linhas, basta selecionar apenas aquelas linhas que você deseja manter:

```
# remove linha 4 do data.frame  
funcionarios <- funcionarios[-4, ]
```

```
# remove linhas em que salario <= 1150  
funcionarios <- funcionarios[funcionarios$salario > 1150, ]
```

## Filtrando linhas com vetores logicos

Relembrando: se passarmos um vetor lógico na dimensão das linhas, selecionamos apenas aquelas que são TRUE. Assim, por exemplo, se quisermos selecionar aquelas linhas em que a coluna salario é maior do que um determinado valor, basta colocar esta condição como filtro das linhas:

```
# Apenas linhas com salario > 1000
funcionarios[funcionarios$salario > 1000, ]
##      nome sexo salario experiencia
## 2 Maria    F    1200           12
## 3  José    M    1300           15

# Apenas linhas com sexo == "F"
funcionarios[funcionarios$sexo == "F", ]
##      nome sexo salario experiencia
## 2 Maria    F    1200           12
```

## Alternativas para `rbind()` e `cbind()` para `data.frames`

As funções `rbind()` e `cbind()` podem não ser muito eficientes. As funções `bind_rows()` e `bind_cols()` do pacote `dplyr` (que veremos mais a frente) são alternativas interessantes. Veremos mais sobre o `dplyr` na aula de manipulações de `data.frames`.

## Funções de conveniência: `subset()`

Uma função de conveniência para selecionar linhas e colunas de um `data.frame` é a função `subset()`, que tem a seguinte estrutura:

```
subset(nome_do_data_frame,  
       subset = expressao_logica_para_filtrar_linhas,  
       select = nomes_das_colunas,  
       drop   = simplificar_para_vetor?)
```

## Funções de conveniência: subset()

Vejamos alguns exemplos:

```
# funcionarios[funcionarios$sexo == "F",]  
subset(funcionarios, sexo == "F")  
##      nome sexo salario experiencia  
## 2 Maria    F    1200           12  
  
# funcionarios[funcionarios$sexo == "M", c("nome", "salario")]  
subset(funcionarios, sexo == "M", select = c("nome", "salario"))  
##      nome salario  
## 3 José    1300
```

## Funções de conveniência: with

A função `with()` permite que façamos operações com as colunas do `data.frame` sem ter que ficar repetindo o nome do `data.frame` seguido de `$`, `[ , ]` ou `[[ ]]` o tempo inteiro. Apenas para ilustrar:

*# Com o with*

```
with(funcionarios, (salario^3 - salario^2)/log(salario))  
## [1] 2.4e+08 3.1e+08
```

*# Sem o with*

```
(funcionarios$salario^3 - funcionarios$salario^2)/log(funcionarios$salario)  
## [1] 2.4e+08 3.1e+08
```

## Funções de conveniência: with

Quatro formas de fazer a mesma coisa (pense em outras formas possíveis):

```
subset(funcionarios, sexo == "M", select = "salario", drop = TRUE)
## [1] 1300
```

```
with(funcionarios, salario[sexo == "M"])
## [1] 1300
```

```
funcionarios$salario[funcionarios$sexo == "M"]
## [1] 1300
```

```
funcionarios[funcionarios$sexo == "M", "salario"]
## [1] 1300
```

## Aplicando funções no data.frame: o que funcionava nas matrizes continua valendo

As funções `rowSums()`, `rowMeans()`, `colSums()`, `colMeans()` e `apply()` continuam funcionando normalmente nos `data.frames`. Teste os seguintes códigos no `data.frame` `df.vendas`:

```
rowSums(df.vendas)
## João Maria Ana
## 5025 4858 6670

colSums(df.vendas)
## segunda  terça  quarta  quinta  sexta
## 3171 2780 3616 3867 3119
```

Funções: definição, argumentos e operadores binários  
Um pouco sobre escopo de funções  
Um pouco sobre métodos de funções

## Funções: definição, argumentos e operadores binários

## Por que funções?

Uma das grandes vantagens de usar uma linguagem de programação é automatizar o seu trabalho ou análise. Você será capaz de realizar grande parte do trabalho utilizando as funções internas do R ou de pacotes de terceiros em um script. Entretanto, você ganha ainda mais flexibilidade e agilidade criando suas próprias funções.

Vejam os um exemplo de *script*:

```
# queremos converter para numeric, retirar os dados discrepantes  
# dividir por 1000 e arredondar o resultado  
precos <- c("0.1", "1250.55", "2346.87", "3467.40", "10000000")  
precos <- as.numeric(precos)  
precos <- precos[!(precos < 1 | precos > 10000)]  
precos <- precos/1000  
precos <- round(precos)  
precos  
## [1] 1 2 3
```

## Por que funções?

Nosso *script* faz o trabalho corretamente. Mas imagine que você queira realizar o mesmo procedimento com um vetor de preços diferente, digamos, `precos2`. Da forma como o seu código foi feito, você terá que **copiar e colar** os comandos e substituir os nomes.

```
# novo vetor de precos
precos2 <- c("0.0074", "5547.85", "2669.98", "8789.45", "150000000")
precos2 <- as.numeric(precos2)
precos2 <- precos2[!(precos2 < 1 | precos2 > 10000)]
precos2 <- precos2/1000
precos2 <- round(precos2)
precos2
## [1] 6 3 9
```

Note como isto é ineficiente. Além do trabalho de copiar e colar todo o seu código para cada análise diferente que você desejar fazer, você ainda estará sujeito a diversos erros operacionais, como esquecer de trocar um dos nomes ao copiar e colar em cada análise.

Funções: definição, argumentos e operadores binários

Um pouco sobre escopo de funções

Um pouco sobre métodos de funções

## Por que funções?

O ideal, aqui, é criar uma **função** que realize este trabalho! Daqui a pouco veremos como fazer isso, primeiramente vejamos como definir uma função no R.

## Definição

Uma função, no R, é definida da seguinte forma:

```
nomeDaFuncao <- function(arg1, arg2, arg3 = default3, ...){  
  # corpo da função: uma série de comandos válidos.  
  return(resultado) # opcional  
}
```

- o comando `function()` diz para o R que você está definindo uma função.
- os valores dentro dos parênteses de `function()` são os argumentos (ou parâmetros) da função. Argumentos podem ter valores default, que são definidos com o sinal de igualdade (no caso `arg3` tem como default o valor `default3`). Existe um parâmetro coringa muito útil, o `...`, o qual veremos mais a frente.
- dentro das chaves encontra-se o “corpo” da função, isto é, uma série de comandos válidos que serão realizados.
- o comando `return()` encerra a função e retorna seu argumento. Como veremos, o `return()` é opcional. Caso omitido, a função retorna o último objeto calculado. ***A função só pode retornar um objeto!***

## Definição

Criemos nossas primeiras funções:

```
quadrado <- function(x){  
  x ^ 2  
}  
quadrado(3)  
## [1] 9  
## se for na mesma linha não precisa das chaves  
quadrado <- function(x) x ^ 2  
quadrado(3)  
## [1] 9  
  
## adicionando mais argumentos  
elevado_n <- function(x,n) x ^ n  
elevado_n(3, 3)  
## [1] 27
```

## Definição

Funções criam um ambiente local e, em geral, ***não alteram diretamente o objeto ao qual são aplicadas***. Elas tomam um objeto como argumento e criam outro objeto, modificado, como resultado. Na maior parte dos casos, a idéia é não ter efeitos colaterais com objetos fora da função.

```
x <- 10
elevado_n(x, 2) # isso altera o valor de x?
## [1] 100
x # não
## [1] 10

# se você quer salvar o resultado
# tem que atribuir a outro objeto
y <- elevado_n(x, 2)
y
## [1] 100
```

## Exercícios

Sua vez.

- Crie uma função `soma_e_subtrai(x, y)` que receba um parâmetro `x` e outro `y` e retorne o resultado tanto da soma  $x + y$  quanto da subtração  $x - y$ . Lembre que uma função no R retorna apenas um objeto, você terá que combinar os resultados (em um vetor, por exemplo) antes de retorná-los.
- Crie os objetos `z <- 1` e `w <- 2`. Teste sua função usando `z` e `w`.

## Exercícios

```
soma_e_subtrai <- function(x, y){  
  soma <- x + y  
  subtracao <- x - y  
  resultado <- c(soma = soma, subtracao = subtracao)  
  return(resultado)  
}  
  
z <- 1  
w <- 2  
  
soma_e_subtrai(z, w)  
##      soma subtracao  
##      3      -1
```

## Voltando ao exemplo

Vamos voltar ao nosso exemplo inicial. Montemos uma função que realiza o tratamento dos dados visto anteriormente:

```
limparDados <- function(dados){  
  dados <- as.numeric(dados)  
  dados <- dados[!(dados < 1 | dados > 10000)]  
  dados <- dados/1000  
  dados <- round(dados)  
  return(dados)  
}  
  
ls() # note que a função foi criada  
## [1] "elevado_n"      "limparDados"    "precos"         "precos2"  
## [5] "quadrado"       "soma_e_subtrai" "w"              "x"  
## [9] "y"              "z"
```

## Voltando ao exemplo

Vejamos em detalhes:

- o comando `function()` diz para o R que você está definindo uma função.
- os valores dentro dos parênteses de `function()` são os argumentos da função. No nosso caso definimos um único argumento chamado `dados`.
- dentro das chaves encontra-se o “corpo” da função, isto é, as operações que serão realizadas. Neste caso, transformamos em `numeric`, retiramos dados menores que 1 e maiores do que 10000, dividimos por 1000 e arredondamos.
- a função `return()` encerra a função e retorna o vetor `dados` modificado.

## Voltando ao exemplo

Pronta a função, sempre que você quiser realizar essas operações em um vetor diferente, basta utilizar `limparDados`.

```
precos3 <- c("0.02", "4560.45", "1234.32", "7894.41", "12000000")
precos4 <- c("0.001", "1500000", "1200.9", "2000.2", "4520.5")
precos5 <- c("0.05", "1500000", "1000000", "7123.4", "9871.5")

# limpando os dados
limparDados(precos3)
## [1] 5 1 8
limparDados(precos4)
## [1] 1 2 5
limparDados(precos5)
## [1] 7 10
```

## Voltando ao exemplo

Bem melhor do que o *script*.

Note que tínhamos 3 vetores diferentes e bastou chamar a função três vezes, ao invés de ter que copiar e colar três vezes o código. Note, também, que se houver algum erro, **temos que corrigir apenas na função**.

E podemos refinar ainda mais `limparDados`.

Por exemplo, da forma como a função está escrita, os valores de corte de mínimo e de máximo serão sempre 1 e 10000; além disso, os resultados sempre serão divididos por 1000. E se quisermos modificar esses valores? Basta colocá-los também como argumentos.

## Mais argumentos

Colocando mais argumentos:

```
limparDados <- function(dados, min, max, div){  
  dados <- as.numeric(dados)  
  dados <- dados[!(dados < min | dados > max)]  
  dados <- dados/div  
  dados <- round(dados)  
  return(dados)  
}
```

Agora você pode alterar os valores de min, max e div ao aplicar a função.

## Mais argumentos

```
precos3 <- c("0.02", "4560", "1234", "7894", "12000000")

limparDados(precos3, min = 0, max = 5000, div = 2)
## [1] 0 2280 617

limparDados(precos3, min = 0, max = 4000, div = 4)
## [1] 0 308

limparDados(precos3, min = -Inf, max = Inf, div = 1)
## [1] 0.0e+00 4.6e+03 1.2e+03 7.9e+03 1.2e+07
```

## Mais argumentos

Note que argumentos são nomeados. Se você colocar os argumentos com seus nomes, a ordem dos argumentos não importa. Entretanto, você pode omitir os nomes dos argumentos, desde que os coloque em ordem correta.

```
# argumentos em ordem diferente  
limparDados(max = 5000, div = 2, min = 0, dados = precos3)  
## [1]      0 2280  617  
  
# argumentos sem nomes (na ordem)  
limparDados(precos3, 0, 4000, 4)  
## [1]      0 308
```

## Argumentos Default

Com mais argumentos, se você esquecer de especificar algum, ocorrerá um erro:

```
limparDados(precos3, max = 5000, div = 1)  
## Error in limparDados(precos3, max = 5000, div = 1): argumento "min" ausente, se
```

Para sanar isto, basta definir valores padrão (default).

## Argumentos Default

Colocando argumentos padrão (default):

```
limparDados <- function(dados, min = 1, max = 10000, div = 1000){  
  dados <- as.numeric(dados)  
  dados <- dados[!(dados < min | dados > max)]  
  dados <- dados/div  
  dados <- round(dados)  
  return(dados)  
}
```

Podemos usar a função omitindo os argumentos que possuem default.

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

Executando código de maneira condicional: `if()`, `if() else`,  
`ifelse()`

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

Há ocasiões em que queremos ou precisamos executar parte do código **apenas se alguma condição for atendida**. Veremos três formas de fazer isso: `if()`, `if() else` e `ifelse()`.

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## O `if()`: estrutura

A estrutura básica:

```
if (condicao) {  
  
  # comandos que  
  # serao rodados  
  # caso condicao = TRUE  
  
}
```

- O início do código se dá com o comando `if` seguido de parênteses e chaves;
- Dentro do parênteses temos uma condição lógica, que deverá ter como resultado ou `TRUE` ou `FALSE`;
- Dentro das chaves temos o bloco de código que será executado se – e somente se – a condição do parênteses for `TRUE`.

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## O `if()`: exemplo

```
# vetores de condição lógica
cria_x <- TRUE
cria_y <- FALSE
# só executa se cria_x = TRUE
if (cria_x) {
  x <- 1
}
# só executa se cria_y = TRUE
if (cria_y) {
  y <- 1
}

# note que x foi criado mas y não
exists("x"); exists("y")
## [1] TRUE
## [1] FALSE
```

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## O `if()` com o `else`

Outra forma de executar códigos de maneira condicional é acrescentar ao `if()` o opcional `else`.  
Estrutura básica:

```
if (condicao) {  
  # comandos que  
  # serao rodados  
  # caso condicao = TRUE  
} else {  
  # comandos que  
  # serao rodados  
  # caso condicao = FALSE  
}
```

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`

Executando código diversas vezes (loops): `for()` e `while()`

A forma do R de fazer loops: voltando à família `apply`!

## O `if()` com o `else`

Em detalhes:

- O início do código se dá com o comando `if` seguido de parênteses e chaves;
- Dentro do parênteses temos uma condição lógica, que deverá ter como resultado ou `TRUE` ou `FALSE`;
- Dentro das chaves do `if()` temos um bloco de código que será executado se – e somente se – a condição do parênteses for `TRUE`.
- Logo em seguida temos o `else` seguido de chaves;
- Dentro das chaves do `else` temos um bloco de código que será executado se – e somente se – a condição do parênteses for `FALSE`.

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## O `if()` com o `else`: exemplo

Como no caso anterior, vejamos primeiramente um exemplo bastante simples.

```
numero <- 1

if (numero == 1) {
  cat("o numero é igual a 1")
} else {
  cat("o numero não é igual a 1")
}

## o numero é igual a 1
```

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## O `if()` com o `else`: encadeando

É possível encadear diversos `if()` `else` em sequência:

```
numero <- 10

if (numero == 1) {
  cat("o numero é igual a 1")
} else if (numero == 2) {
  cat("o numero é igual a 2")
} else {
  cat("o numero não é igual nem a 1 nem a 2")
}
## o numero não é igual nem a 1 nem a 2
```

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## Exemplo: par ou ímpar?

Vamos criar uma função que nos diga se um número é par ou ímpar. Nela vamos utilizar tanto o `if()` sozinho quanto o `if() else`. Vale lembrar que um número (inteiro) é par se for divisível por 2 e que podemos verificar isso se o resto da divisão (operador `%` no R) deste número por 2 for igual a zero.

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## Exemplo: par ou ímpar?

```
par_ou_impar <- function(x){  
  # verifica se o número é um decimal comparando o tamanho da diferença de x e floor(x)  
  # se for decimal retorna NA (pois par e ímpar não fazem sentido para decimais)  
  if (abs(x - round(x)) > 1e-7) {  
    return(NA)  
  }  
  
  # se o número for divisível por 2 (resto da divisão zero) retorna "par"  
  # caso contrário, retorna "ímpar"  
  if (x %% 2 == 0) {  
    return("par")  
  } else {  
    return("impar")  
  }  
}
```

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## Exemplo: par ou ímpar?

Testando:

```
par_ou_impar(4)
## [1] "par"
```

```
par_ou_impar(5)
## [1] "impar"
```

```
par_ou_impar(2.1)
## [1] NA
```

Parece que está funcionando bem, mas...

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## Lembre-se: `if()` não é vetorizado.

Há um pequeno problema:

```
x <- 1:5
par_ou_impar(x)
## Warning in if (abs(x - round(x)) > 1e-07) {: a condição tem comprimento > 1 e
## somente o primeiro elemento será usado
## Warning in if (x%%2 == 0) {: a condição tem comprimento > 1 e somente o primeir
## elemento será usado
## [1] "impar"
```

Provavelmente não era isso o que esperávamos. O que está ocorrendo aqui?

Os comandos `if()` e `if() else` **não são vetorizados**.

O `if()` aceita apenas um único valor, seja `TRUE` ou `FALSE`. Se você passar mais de um valor, ele ignora os demais e usa apenas o primeiro.

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## Lembre-se: `if()` não é vetorizado.

Revedo a questão em um exemplo mais simples, note que o `if()` irá usar apenas o primeiro valor do vetor `c(F, T)`:

```
if (c(F, T)) {  
  
  print("TRUE")  
  
} else {  
  
  "FALSE"  
}  
## Warning in if (c(F, T)) {: a condição tem comprimento > 1 e somente o p:  
## elemento será usado  
## [1] "FALSE"
```

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## A função `ifelse()`

Uma alternativa para casos como esses é utilizar a função `ifelse()`.  
Estrutura básica:

```
ifelse(vetor_de_condicoes, valor_se_TRUE, valor_se_FALSE)
```

- o primeiro argumento é um vetor (ou uma expressão que retorna um vetor) com vários TRUE e FALSE;
- o segundo argumento é o valor que será retornado quando o elemento do `vetor_de_condicoes` for TRUE;
- o terceiro argumento é o valor que será retornado quando o elemento do `vetor_de_condicoes` for FALSE.

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## A função `ifelse()`: exemplo

Primeiramente, vejamos um caso trivial:

```
ifelse(c(TRUE, FALSE, FALSE, TRUE), 1, -1)
## [1] 1 -1 -1 1
```

Note que passamos um vetor de condições com `TRUE`, `FALSE`, `FALSE` e `TRUE`.  
O valor para o caso `TRUE` é 1 e o valor para o caso `FALSE` é -1.  
Logo, o resultado é 1, -1, -1 e 1.

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## A função `ifelse()`: voltando ao par ou ímpar

Vamos criar uma versão com `ifelse` da nossa função que nos diz se um número é par ou ímpar.

```
par_ou_impar_ifelse <- function(x){  
  
  # se x for decimal, retorna NA, se não for, retorna ele mesmo (x)  
  x <- ifelse(abs(x - round(x)) > 1e-7, NA, x)  
  
  # se x for divisível por 2, retorna 'par', se não for, retorna impar  
  ifelse(x %% 2 == 0, "par", "impar")  
}
```

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## A função `ifelse()`: voltando ao par ou ímpar

Perceba que agora a função funciona sem problemas com vetores:

```
par_ou_impar_ifelse(x)
## [1] "impar" "par"   "impar" "par"   "impar"

par_ou_impar_ifelse(c(x, 1.1))
## [1] "impar" "par"   "impar" "par"   "impar" NA
```

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

Executando código diversas vezes (loops): `for()` e `while()`

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## Loops com `for()`

Um loop utilizando `for()` no R tem a seguinte estrutura básica:

```
for(i in conjunto_de_valores){  
  # comandos que  
  # serão repetidos  
}
```

- O início do loop se dá com o comando `for` seguido de parênteses e chaves;
- Dentro do parênteses temos um indicador que será usado durante o loop (no caso escolhemos o nome `i`) e um conjunto de valores que será iterado (`conjunto_de_valores`).
- Dentro das chaves temos o bloco de código que será executado durante o loop.

Em outras palavras, no comando acima estamos dizendo que para cada elemento `i` contido no `conjunto_de_valores` iremos executar os comandos que estão dentro das chaves.

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## Loops com `for()`: exemplo

Vamos imprimir na tela os números de 1 a 5.

```
for (i in 1:5) {  
  print(i)  
}
```

```
## [1] 1  
## [1] 2  
## [1] 3  
## [1] 4  
## [1] 5
```

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## Loops com `for()`: exemplo

Agora, vamos imprimir na tela as 5 primeiras letras do alfabeto (o R já vem com um vetor com as letras do alfabeto: `letters`).

```
for (i in 1:5) {  
  print(letters[i])  
}  
## [1] "a"  
## [1] "b"  
## [1] "c"  
## [1] "d"  
## [1] "e"
```

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## Loops com `for()`: exemplo

No mesmo exemplo, ao invés correr o loop no índice de inteiros `1:5`, vamos iterar diretamente sobre os primeiros 5 elementos do vetor `letters`:

```
for (letra in letters[1:5]) {  
  print(letra)  
}  
## [1] "a"  
## [1] "b"  
## [1] "c"  
## [1] "d"  
## [1] "e"
```

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## Loops com `for()`: `seq_along()` e `length()`

Uma função bastante útil ao fazer loops é a função `seq_along()`. Ela cria um vetor de inteiros com índices para acompanhar o objeto.

```
# criando um vetor de exemplo
set.seed(119)
x <- rnorm(10)

# inteiros de 1 a 10
seq_along(x)
## [1] 1 2 3 4 5 6 7 8 9 10
```

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## Loops com `for()`: `seq_along()` e `length()`

Também é possível criar um vetor de inteiros do tamanho do objeto fazendo uma sequência de 1 até `length(x)`:

```
1:length(x)
## [1] 1 2 3 4 5 6 7 8 9 10
```

Entretanto, a vantagem de `seq_along()` é que quando o vetor é vazio, ela retorna um vetor vazio e, deste modo, o loop não é executado (o que é o comportamento correto). Já a sequência `1:length(x)` retorna a sequência `1:0`, isto é, uma sequência decrescente de 1 até 0, e loop é executado nestes valores. Vejamos.

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## Loops com `for()`: `seq_along()` e `length()`

```
# cria vetor vazio
x <- numeric(0)

# 1:length(x)
# note que o loop é executado (o que é errado)
for (i in 1:length(x)) print(i)
## [1] 1
## [1] 0

# seq_along
# note que o loop não é executado (o que é correto)
for (i in seq_along(x)) print(i)
```

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## Vetorização, funções nativas e loops

- Como vimos, o R é vetorizado. Muitas vezes, quando você pensar que precisa usar um loop, ao pensar melhor, descobrirá que não precisa. Em geral é possível resolver o problema de maneira vetorizada e usando funções nativas do R.
- Para quem está aprendendo a programar diretamente com o R, isso é algo que virá naturalmente. Todavia, para quem já sabia programar em outras linguagens de programação – como C – pode ser difícil se acostumar a pensar desta maneira.

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## Vetorização, funções nativas e loops

Vejamos um exemplo trivial. Suponha que você queira dividir os valores de um vetor `x` por 10. Se o R não fosse vetorizado, você teria que fazer algo como:

```
# criando vetor de exemplo
x <- 10:20

# divide cada elemento por 10
for (i in seq_along(x))
  x[i] <- x[i]/10

# resultado
x
## [1] 1.0 1.1 1.2 1.3 1.4 1.5 1.6 1.7 1.8 1.9 2.0
```

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## Vetorização, funções nativas e loops

Mas o R é vetorizado e, portanto, este é o tipo de loop **que não faz sentido** na linguagem. É muito mais rápido e fácil de entender escrever simplesmente `x/10`, como já tínhamos aprendido nas primeiras aulas!

```
# criando vetor de exemplo
x <- 10:20

# divide cada elemento por 10
x <- x/10
x
## [1] 1.0 1.1 1.2 1.3 1.4 1.5 1.6 1.7 1.8 1.9 2.0
```

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## Vetorização, funções nativas e loops

Vejamos um caso um pouco mais complicado. Suponha que você queira, gerar um passeio aleatório com um algoritmo simples: a cada período você pode andar para frente (+1) ou para trás (-1) com probabilidades iguais.

```
set.seed(1)

# número de passos
n <- 1000
# vetor para armazenar o passeio aleatório
passeio <- numeric(n)
# primeiro passo
passeio[1] <- sample(c(-1, 1), 1)
# demais passos
for (i in 2:n) {
  # passo i é o onde você estava (passeio[i-1]) mais o passo seguinte
  passeio[i] <- passeio[i - 1] + sample(c(-1, 1), 1)
}
```

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## Vetorização, funções nativas e loops

É possível fazer tudo isso com apenas uma linha de maneira “vetorizada” e bem mais eficiente: crie todos os `n` passos de uma vez e faça a soma acumulada.

```
set.seed(1)
passeio2 <- cumsum(sample(c(-1, 1), n, TRUE))

# verifica se são iguais
all.equal(passeio, passeio2)
## [1] TRUE
```

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`

Executando código diversas vezes (loops): `for()` e `while()`

A forma do R de fazer loops: voltando à família `apply`!

## Vetorização, funções nativas e loops

Então, você deve estar se perguntando: “não é para usar loops nunca”?

Não é isso. Em algumas situações loops são inevitáveis e podem inclusive ser mais fáceis de ler e de entender. O ponto aqui é apenas lembrá-lo de explorar a vetorização do R.

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## DICA: pré-alocar espaço antes do loop

Um erro bastante comum de quem está começando a programar em R é “crescer” objetos durante o loop. Isto tem um impacto substancial na performance do seu programa!

- Sempre que possível, crie um objeto, antes de iniciar o loop, para armazenar os resultados de cada iteração.

Vamos calcular os  $n$  primeiros números da sequência de Fibonacci:

$F_1 = 0, F_2 = 1, F_3 = 1, F_4 = 2, F_5 = 3, F_6 = 5, F_7 = 8, F_8 = 13, F_9 = 21 \dots$  Note que a sequência de Fibonacci pode ser definida da seguinte forma:

- os primeiros dois números são 0 e 1, isto é,  $F_1 = 0, F_2 = 1$ ;
- A partir daí, os números subsequentes são a soma dos dois números anteriores, isto é,  $F_i = F_{i-1} + F_{i-2}$  para todo  $i > 2$ .

Vejamos uma forma de implementar isto no R usando `for()` e criando um vetor para armazenar os resultados.

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## DICA: pré-alocar espaço antes do loop

```
n <- 9
# crie um vetor de tamanho n
# para armazenar os n resultados
fib <- numeric(n)
# comece definindo as condições iniciais
# F1 = 0 e F2 = 1
fib[1] <- 0
fib[2] <- 1
# Agora para todo i > 2
# calculamos Fi = F(i-1) + F(i - 2)
for (i in 3:n) {
  fib[i] <- fib[i - 1] + fib[i - 2]
}
# conferindo resultado
fib
## [1] 0 1 1 2 3 5 8 13 21
```

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## Loops com `while()`

Estrutura básica:

```
while (condicao) {  
  # código a ser executado  
  # até que condição seja TRUE  
  # em geral a condição será atualizada  
  # dentro do código  
}
```

- O início do loop se dá com o comando `while` seguido de parênteses e chaves;
- Dentro do parênteses temos uma condição lógica que será testada;
- Enquanto a condição lógica for verdadeira, bloco de código que está entre chaves será executado repetidamente.

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## Loops com `while()`: exemplo

Lembra que com o `for` contamos de 1 até 5? Como fazer a mesma coisa com o `while`?

```
# i inicial
i <- 1
# condição:
# enquanto i for menor ou igual a 5
while (i <= 5) {
  print(i)
  # atualiza i (cuidado com loop infinito!)
  i <- i + 1
}
## [1] 1
## [1] 2
## [1] 3
## [1] 4
## [1] 5
```

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## loop infinito

Se você entrar em um loop infinito, aperte `esc` para sair. Vamos testar: o código abaixo irá iniciar um loop infinito incrementando a variável `i`. Deixe o código rodando por um tempo e, depois, aperte `esc` para interromper.

```
i <- 1
while (i>=1) {
  print(i)
  i <- i + 1
}
```

- Por que o código acima está em loop infinito?

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`

Executando código diversas vezes (loops): `for()` e `while()`

A forma do R de fazer loops: voltando à família `apply`!

## Qual a diferença? `while()` vs `for()`

Usar o `while()` para contar de 1 até 5 é bem mais complicado do que com o `for()`, então qual a vantagem do `while()`? O `while()` é fundamental quando precisamos rodar uma função repetidas vezes **mas não sabemos quantas** vezes!

```
set.seed(1)
x <- rnorm(1)
i <- 1

while (x < 2) {
  i <- i + 1
  x <- rnorm(1)
}

# quantas vezes rodou?
i
## [1] 61
```

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## Mais controle sobre loops: `break()`

A função `break()` interrompe a execução de um loop no momento em que é chamada.

- exemplo: quero que o loop rode de 1 até 10, mas se por acaso a variável `u` for maior do que 0.8, o loop é interrompido.

```
set.seed(25)
for (i in 1:10) {
  u <- runif(1)
  if (u > 0.8) break()
  print(i)
}
## [1] 1
## [1] 2
## [1] 3
```

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## Mais controle sobre loops: `next()` e `break()`

A função `next()` faz com que o loop passe para a próxima iteração no momento em que é chamada.

- exemplo: quero que o loop rode de 1 até 10, mas se por acaso a variável `u` for maior do que 0.5, eu pulo aquela parte do loop.

```
set.seed(25)
for (i in 1:10) {
  u <- runif(1)
  if (u > 0.5) next()
  print(i)
}
## [1] 1
## [1] 3
## [1] 5
## [1] 8
## [1] 9
## [1] 10
```

Executando código de maneira condicional: `if()`, `if() else`, `ifelse()`  
Executando código diversas vezes (loops): `for()` e `while()`  
A forma do R de fazer loops: voltando à família `apply`!

## Outra forma de fazer `while()`: `repeat` + `break`

A função `repeat` repete o código entre chaves indefinidamente. Deve ser utilizada **sempre** em conjunto com a função `break()`.

```
# contando de 1 até 5
i <- 1
repeat {
  print(i)
  i <- i + 1
  if (i > 5) break()
}
## [1] 1
## [1] 2
## [1] 3
## [1] 4
## [1] 5
```